
SECURE SOFTWARE ARCHITECTURE

Software Engineering HSC Course



WHAT IS SECURE SOFTWARE ARCHITECTURE?

Secure software architecture refers to the structure and design of a software system that incorporates security measures to protect against cyber threats and vulnerabilities.

BENEFITS OF DEVELOPING SECURE SOFTWARE

```
mirror_mod = modifier_ob.  
Set mirror object to mirror.  
mirror_mod.mirror_object =
```

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1
```

```
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly
```

-- OPERATOR CLASSES --

```
types.Operator):  
    "X mirror to the selected  
    object.mirror_mirror_x"  
    "mirror X"
```

```
context):  
    context.active_object is not
```

1. DATA PROTECTION

- **Data protection** refers to safeguarding personal, sensitive, or confidential data from unauthorised access, corruption, or loss.
 - **Benefits:**
 - **Confidentiality:** Ensures only authorized users can access data.
 - **Integrity:** Prevents unauthorized modification of data.
 - **Availability:** Ensures data is accessible when needed.
-

EXAMPLE:

A hospital management system stores patient records. If the software is secure:

- Only doctors and nurses can access patient files.
 - Records cannot be altered without logging the change.
 - Backups ensure data is not lost in case of system failure.
-

2. MINIMISING CYBER ATTACKS AND VULNERABILITIES

- **Definition:** Secure software reduces the risk of being exploited by cybercriminals through vulnerabilities in the code.
 - **Benefits:**
 - Prevents **unauthorised access**, **data breaches**, and **malware infections**.
 - Reduces the **attack surface** (the number of ways an attacker can enter the system).
 - Ensures **system stability** and **user trust**.
-



EXAM READY SUMMARY

Data Protection

Developing secure software protects sensitive data from unauthorised access, theft, or corruption. Using encryption, authentication, and access controls ensures data integrity and privacy, meeting legal requirements and building user trust. Strong data protection reduces the risk of breaches, financial loss, and reputational damage.

Minimising Cyber Attacks and Vulnerabilities

Secure software reduces cyber attacks by addressing vulnerabilities like weak authentication and code exploits. Applying secure coding practices, input validation, and regular updates prevents hackers from exploiting flaws. This proactive approach maintains system reliability, avoids costly downtime, and ensures long-term resilience against evolving threats.

EXAM READY SUMMARY 2



Developing secure software is essential for protecting sensitive data and reducing cyber threats. Strong security measures such as encryption, authentication, and access controls safeguard personal and financial information, ensuring compliance with privacy laws and building user trust. At the same time, secure coding practices, input validation, and regular updates minimise vulnerabilities that hackers exploit, preventing attacks like ransomware and denial-of-service. This proactive approach reduces financial loss, downtime, and reputational damage.

INTERPRET AND APPLY FUNDAMENTAL SOFTWARE DEVELOPMENT STEPS TO DEVELOP SECURE CODE



SOFTWARE DEVELOPMENT STEPS



REQUIREMENTS
DEFINITION



DETERMINING
SPECIFICATIONS



DESIGN



DEVELOPMENT



INTEGRATION



TESTING AND
DEBUGGING



INSTALLATION



MAINTENANCE

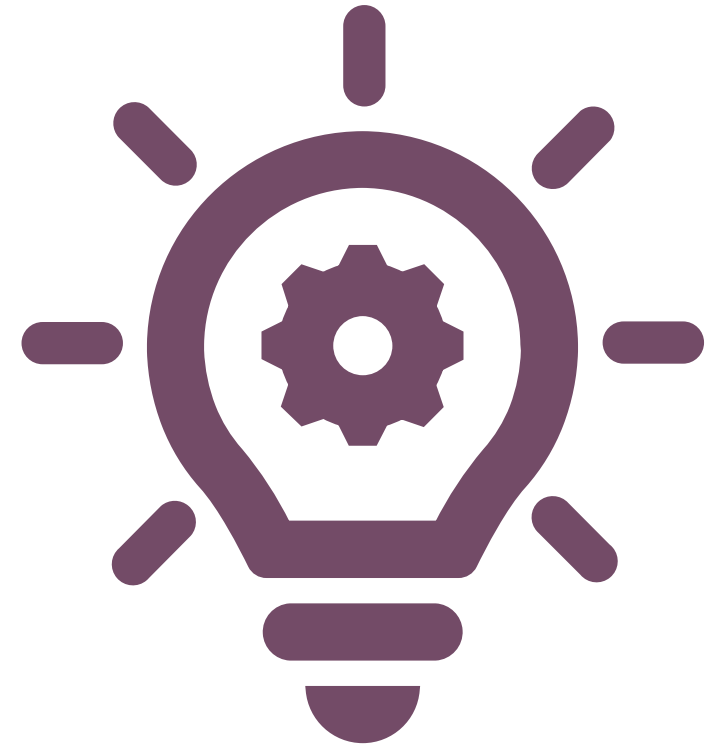


REQUIREMENTS DEFINITION

- Identify **functional** and **security requirements** before coding.
- Consider what data needs protection.
- Identify **threats** and **risks** (e.g., unauthorised access, data loss).
- Determine legal and compliance obligations

Example:

In a school portal, security requirements include encrypted student data and role-based access for teachers and admin.



DETERMINING SPECIFICATIONS

- Translate requirements into **measurable specifications**.
- Define security standards (e.g., password length, encryption type).
- Specify **user access control levels** and **data handling rules**.
- Set measurable outcomes for testing security (e.g., “system must lock after 3 failed logins”).

Example:

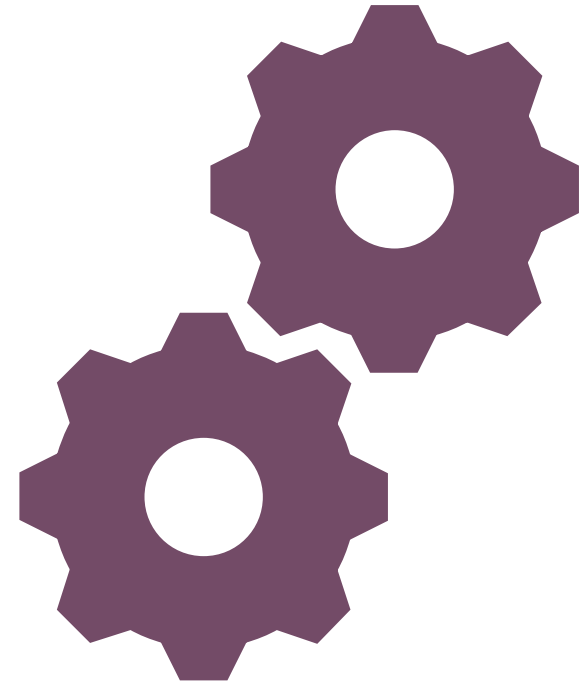
Login must require multi-factor authentication (MFA) and session timeout after inactivity.

DESIGN

- Apply **secure-by-design principles**:
 - Least privilege
 - Defence in depth
 - Secure defaults
- Map data flow diagrams to identify where information could be exposed.
- Incorporate **encryption, authentication, and privacy-by-design** early.

Example:

Designing an e-commerce system with a separate payment gateway to isolate sensitive data.

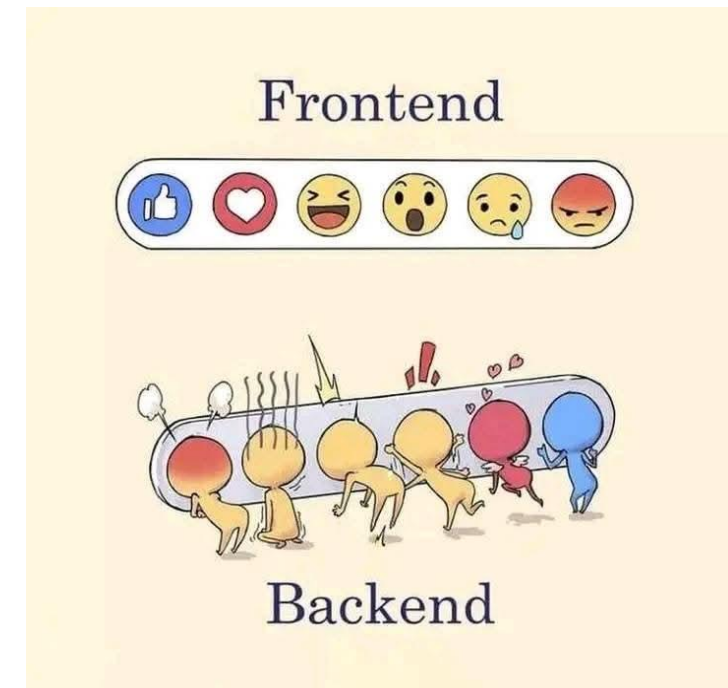


DEVELOPMENT

- Follow **secure coding standards** (e.g., OWASP guidelines).
- Validate and sanitise all user input.
- Avoid hardcoding passwords or keys.
- Use proper **error handling** — don't reveal system info.
- Encrypt sensitive data before storing or transmitting.
- This is where most vulnerabilities are introduced — reinforce testing during development.

Example:

In Python, hash passwords instead of storing them in plain text.



INTEGRATION

- Combine modules and verify secure interaction.
- Use **API authentication** and encryption for communication between systems.
- Ensure consistent security policies across subsystems.
- Scan dependencies for known vulnerabilities (e.g., outdated libraries).

Example:

When integrating a third-party payment service, use HTTPS and token-based authentication.



TESTING AND DEBUGGING

- Perform **functional testing** + **security testing**.
 - Use tools to check for:
 - Injection attacks
 - Broken authentication
 - Cross-site scripting (XSS)
 - Conduct **penetration testing** or code review.
 - Debug without exposing sensitive data in logs.
 - **Example:**
Use **penetration testing** tools to simulate a cyberattack and detect vulnerabilities.
-

INSTALLATION

- Use **secure deployment environments**.
- Apply correct **file permissions** and **access controls**.
- Ensure **HTTPS certificates** and server security settings are active.
- Remove unnecessary default accounts and debug information.
- Train users on secure usage.
- **Example:**
Before launch, the web server is configured with HTTPS and updated TLS certificates.

MAINTENANCE

- Regularly apply security patches and updates.
- Monitor system logs for unusual activity.
- Review and update access controls.
- Conduct **post-deployment vulnerability assessments**.
- Plan for disaster recovery and data backups.

Example:

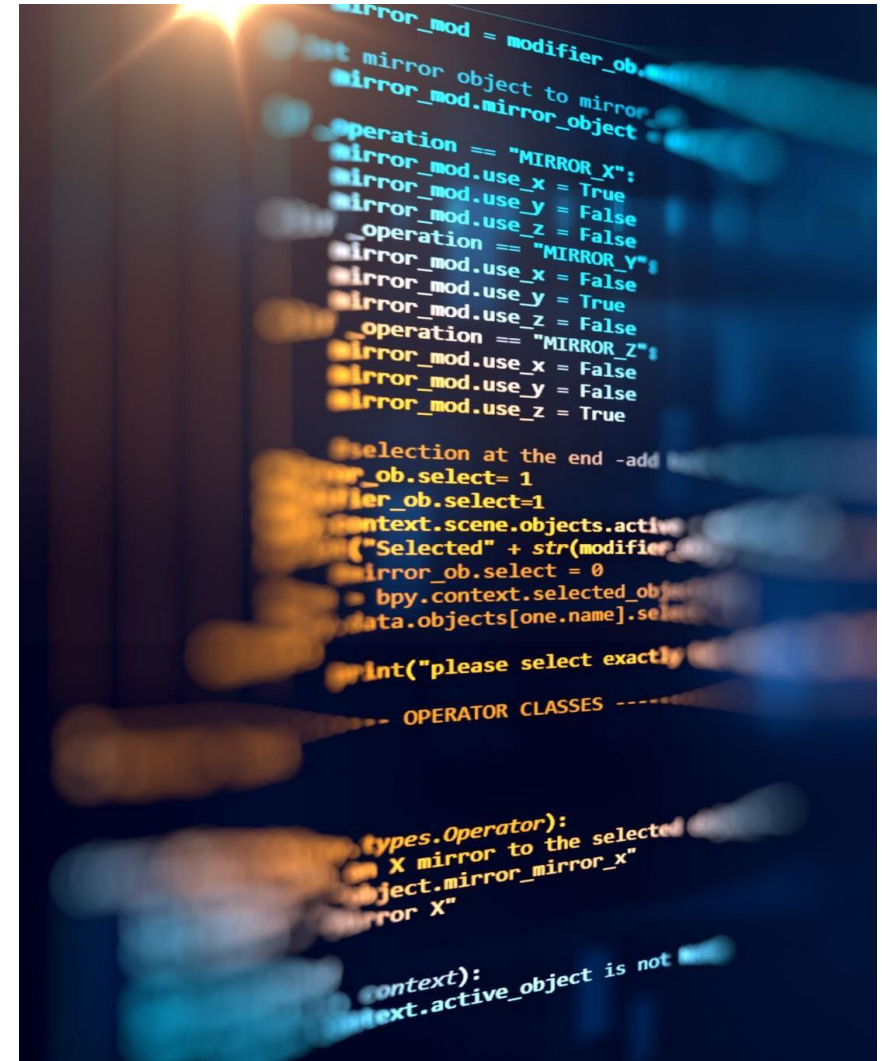
Regular patching of software libraries prevents known exploits

DESCRIBE
HOW THE CAPABILITIES
AND EXPERIENCE OF END
USERS INFLUENCE THE
SECURE DESIGN
FEATURES OF SOFTWARE

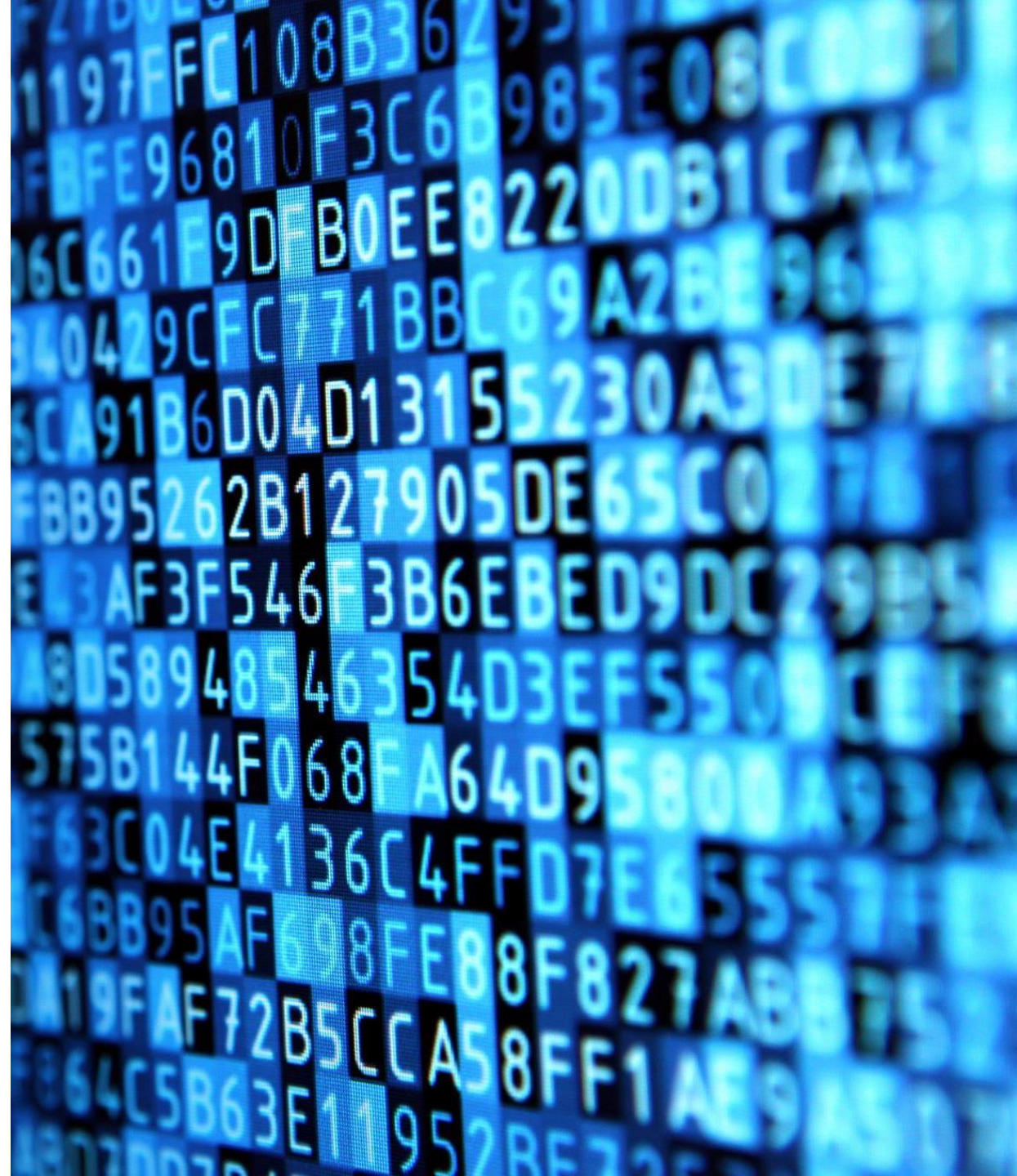


EXAM READY SUMMARY

End-user capabilities strongly influence secure software design, especially in user registration and authentication. If users have limited technical skills, complex registration processes can lead to weak passwords, so developers include features like password strength indicators and clear instructions. For example, an online shopping site may show a real-time password strength meter and suggest adding symbols for stronger credentials. Authentication must also balance security and usability; overly complex logins can cause risky behaviors like writing down passwords. A banking app might use multi-factor authentication (MFA) with a password and fingerprint for high security, while a social media platform may use two-step authentication with a password and emailed code for simplicity. Considering user experience ensures security features are effective and accessible, reducing risks such as phishing and credential reuse.



DEVELOPING SECURE CODE



**EXPLORE FUNDAMENTAL
SOFTWARE DESIGN SECURITY
CONCEPTS WHEN DEVELOPING
PROGRAMMING CODE**

FUNDAMENTAL SOFTWARE DESIGN SECURITY CONCEPTS



CONFIDENTIALITY



INTEGRITY



AVAILABILITY



AUTHENTICATION



AUTHORISATION



ACCOUNTABILITY



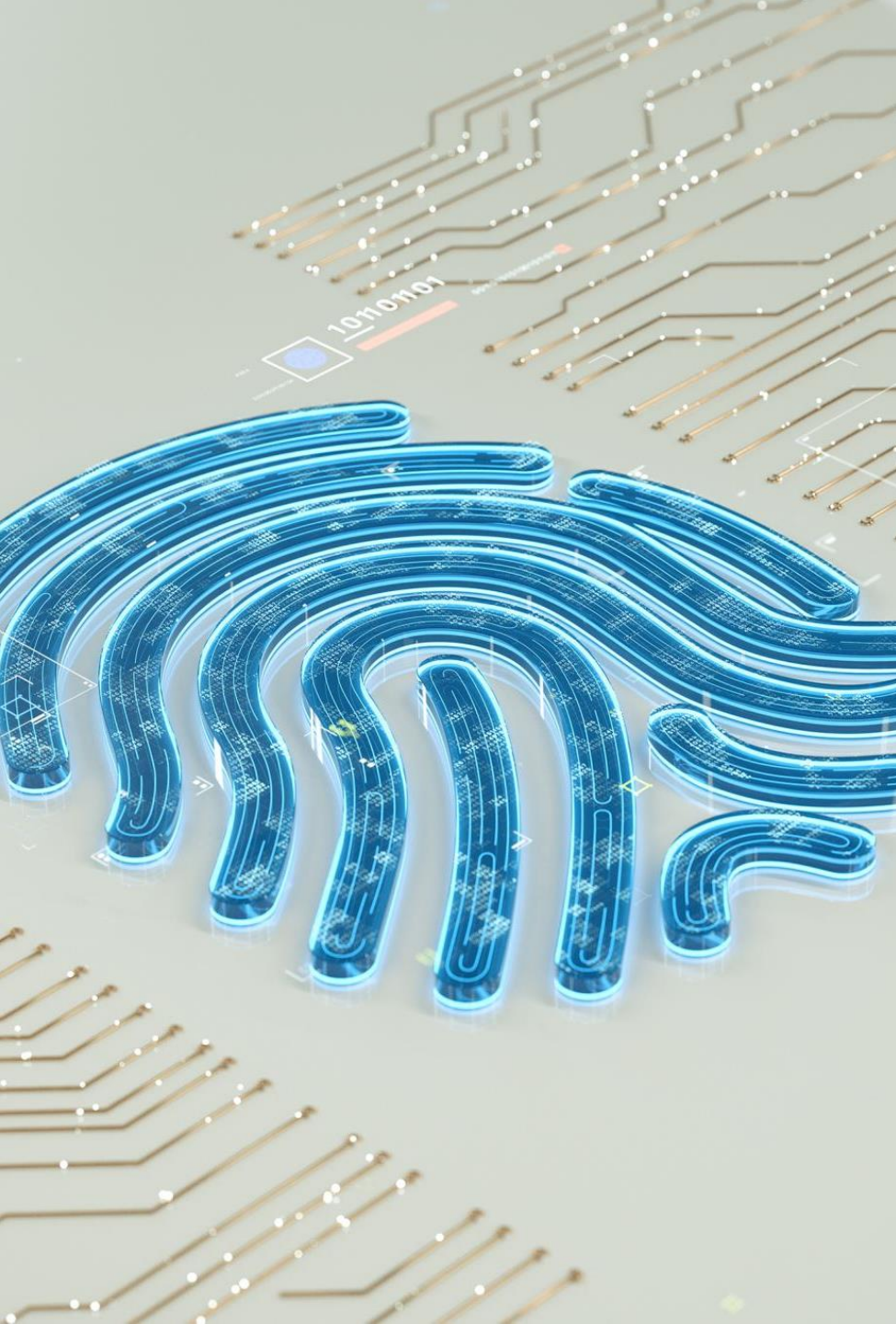
CONFIDENTIALITY

Confidentiality ensures that sensitive information remains accessible only to authorised individuals or processes. In other words, no one else should be able to see or intercept the data.

- **Why It's Important:** Without confidentiality, personal details, trade secrets, or financial data could be exposed or stolen. This can lead to identity theft, loss of competitive advantage, or severe legal consequences.
- **Example of How to Ensure It:**
 - **Encryption** for both data being transmitted and stored on disk (at rest).

INTEGRITY

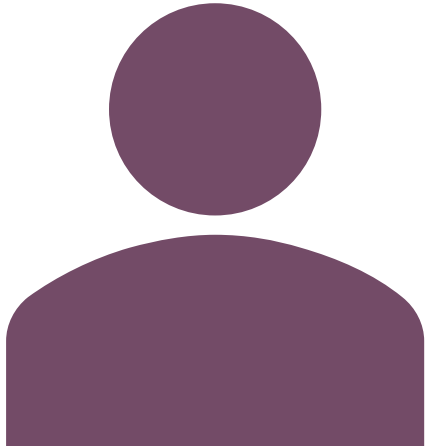
- Integrity means keeping data accurate and unaltered. This is a concern during storage, transmission, or processing.
 - **Why It's Important:** If data can be modified without detection, decisions made using that data become unreliable. This can lead to system malfunctions, incorrect data, or misleading results.
 - **Example of How to Ensure It:**
 - **Hashes** to verify that no changes have been made.
 - **Checksums** to verify that transmitted data is correct.
-



AUTHENTICATION

- Authentication confirms the identity of a user or system, typically via passwords, tokens and even biometrics.
- **Why It's Important:** Strong authentication methods prevent unauthorised access, reducing the risk of impersonation or brute force attacks.
- **Examples of How to Ensure It:**
 - **Multi-Factor Authentication (MFA)** combining passwords with codes, biometrics, etc.
 - **Encrypt passwords in transit** so that they cannot be intercepted.

AUTHORISATION



- Authorisation dictates what actions an authenticated user or system can perform once identity is verified. It ensures users only access and modify resources they have permission to use.
 - **Why It's Important:** Even valid users can pose a risk if they have excessive permissions. Tight control reduces the chance of accidental or malicious data breaches.
 - **Examples of How to Ensure It:**
 - **Role-Based Access Control (RBAC)** that assign privileges according to roles.
 - **Principle of Least Privilege** i.e. giving users the minimum access needed to perform their tasks.
-

AVAILABILITY



AVAILABILITY



Definition:

Availability refers to the ability of a system, service, or application to remain operational and accessible whenever users need it. It is a key component of system reliability.

KEY CONCEPTS



Outage: A period when a service or software is unavailable. **Unplanned Outage:** Caused by failures, crashes, or unexpected issues.



Planned Outage: Scheduled downtime for maintenance, updates, or system changes.

WHY AVAILABILITY MATTERS



Healthcare: Patient records and emergency systems.



E-commerce: Online transactions and customer access.



Internal Services: Payroll, communication tools.

IMPACT OF POOR AVAILABILITY



Financial losses.



Reputational damage.



Operational disruption.



STRATEGIES TO ENSURE AVAILABILITY

- Redundancy & Failover Systems:**

- Backup servers or mirrored sites.
- Automatic failover when primary system fails.

- Load Balancing:**

- Distributes traffic across multiple servers.
- Prevents overload and improves performance.

- Planned Maintenance:**

- Schedule and announce outages.
 - Minimize user impact through clear communication.
-



ACCOUNTABILITY

ACCOUNTABILITY



- **Definition:**
Accountability ensures that every action or event in a system can be traced back to a specific user or process. It is essential for security, compliance, and troubleshooting.

KEY CONCEPTS:

- **Audit Trail:** A chronological record of system activities.
- **Logs:** Detailed records of user actions, system changes, and security events.



WHY ACCOUNTABIL ITY MATTERS:



Problem Resolution: Identify root causes of issues.



Compliance: Meet legal and regulatory requirements.



Deterrence: Discourages misuse and malicious activity.

METHODS TO ENSURE ACCOUNTABILITY

Robust Logging:

- Capture user actions, system changes, and security events.
- Store logs securely to prevent tampering.

Auditing Procedures:

- Regular reviews of logs.
- Automated alerts for suspicious behaviour.

Access Control:

- Ensure only authorized users can perform critical actions.

AVAILABILITY SUMMARY

Availability refers to the ability of a software system or service to remain operational and accessible whenever users need it. It is a critical aspect of reliability because outages—whether planned for maintenance or unplanned due to failures—can disrupt essential operations such as healthcare systems, e-commerce platforms, and internal organizational services. Poor availability can lead to financial losses and reputational damage. To maintain high availability, strategies include implementing redundancy and failover systems to provide backup servers or sites, using load balancing to distribute traffic across multiple servers, and scheduling planned outages with prior announcements to minimize user impact.

ACCOUNTABILITY

Accountability ensures that every action or event within a system can be traced back to a specific user or process. This is achieved through mechanisms such as logs and audit trails, which record user actions, system changes, and security events. Accountability is important for troubleshooting issues, conducting investigations, and meeting legal or regulatory compliance requirements. It also acts as a deterrent against misuse. Key practices include robust logging to capture detailed system activity, auditing procedures such as periodic log reviews and automated alerts for suspicious behavior, and implementing access control to restrict critical actions to authorized users.



HSC STYLE QUESTIONS

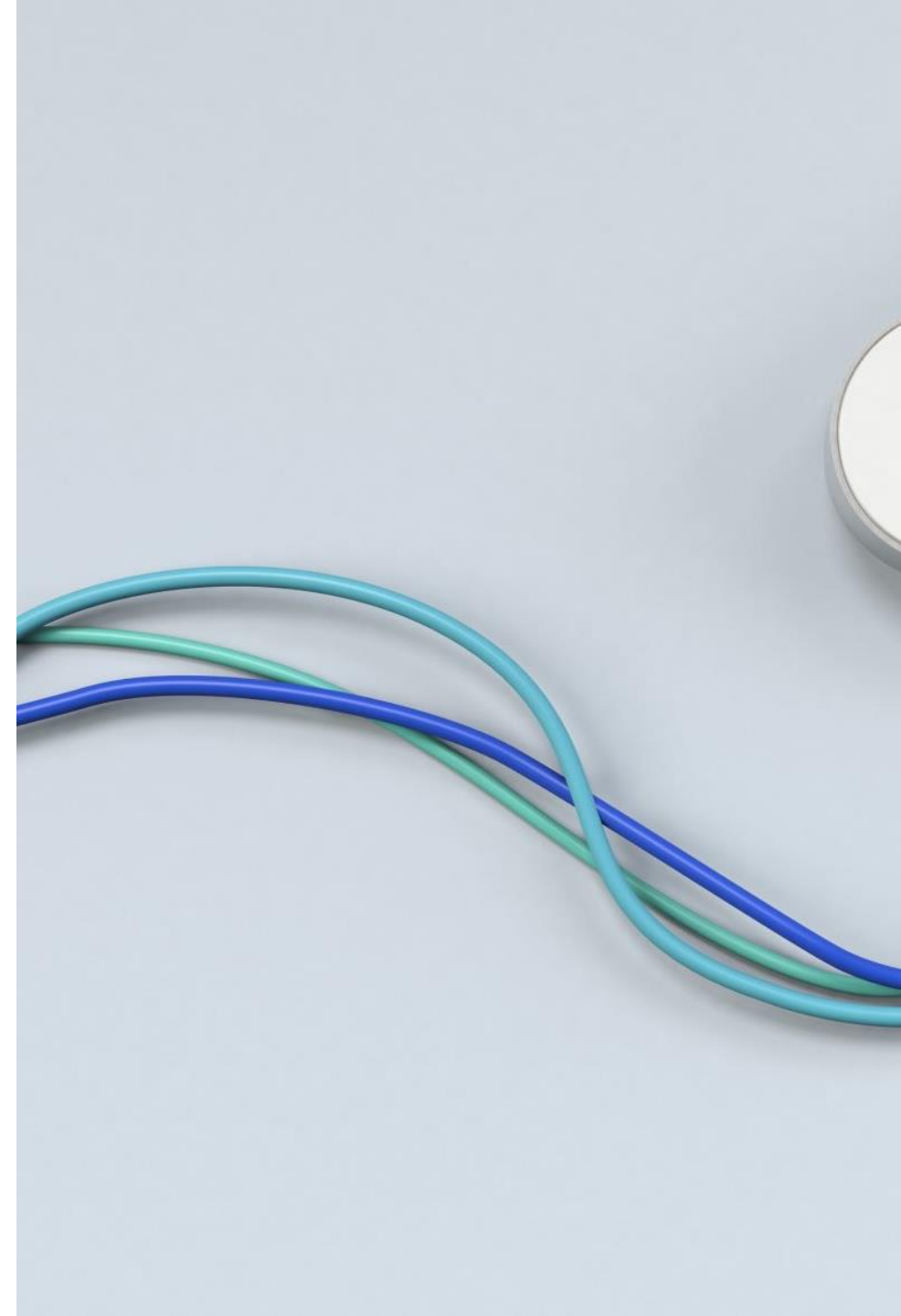
EXPLAIN HOW **CONFIDENTIALITY** AND **INTEGRITY** CAN BE MAINTAINED WHEN DEVELOPING A SCHOOL DATABASE APPLICATION.

Confidentiality can be maintained in a school database application by ensuring that student records are accessible only to authorised staff. This can be achieved through encryption of sensitive information such as addresses, contact details, and medical data, as well as enforcing secure logins for teachers and administrators. Role-based access control ensures that users see only the information relevant to their responsibilities. Integrity is maintained by validating all data entered into the system, preventing errors or malicious alterations. Techniques such as database constraints, input sanitisation, and transaction controls help ensure that stored records remain accurate and consistent over time. Together, these measures protect both the privacy and reliability of student information.

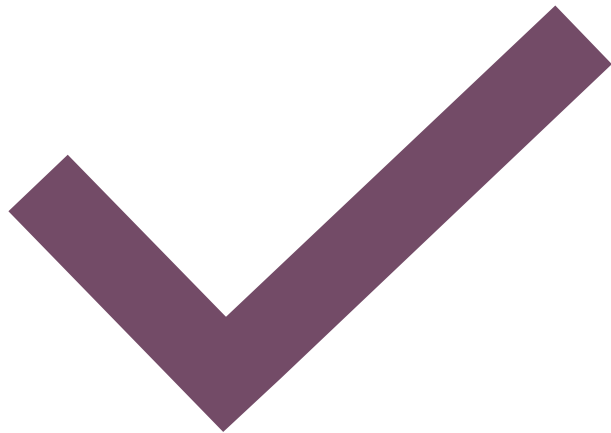


EXPLAIN HOW AUTHENTICATION, AUTHORISATION, AND ACCOUNTABILITY WORK TOGETHER TO PROTECT A CLOUD-BASED FILE-SHARING SYSTEM.

In a cloud-based file-sharing system, authentication, authorisation, and accountability form a layered defence against unauthorised access and misuse. Authentication verifies user identity using secure credentials such as usernames, passwords, or multi-factor authentication. Once authenticated, authorisation determines what actions a user is permitted to perform — for instance, some users may upload and edit files, while others can only view or download them. Accountability ensures all actions are traceable through activity logs and audit trails, allowing administrators to monitor who accessed, modified, or shared files. These three mechanisms work together to confirm user identity, restrict access to appropriate resources, and record activity for transparency and investigation, thereby ensuring the security and integrity of the file-sharing environment.



EXPLAIN HOW THE SIX CORE SECURITY CONCEPTS — CONFIDENTIALITY, INTEGRITY, AVAILABILITY, AUTHENTICATION, AUTHORISATION, AND ACCOUNTABILITY — CAN BE APPLIED WHEN DEVELOPING A SECURE E-COMMERCE PLATFORM.



the **given order** (CIA + AAA) is technically correct but not the most logical for writing a clear, flowing HSC response.

A better order is to group them **conceptually** — from **user access** → **data protection** → **system reliability** → **oversight**.

This structure mirrors the **real-world flow of secure interaction** in a software system:

- **Authenticate** the user →
 - **Authorise** what they can do →
 - **Protect confidentiality** of the data they access →
 - **Preserve integrity** of data and transactions →
 - **Maintain availability** of services for valid users →
 - **Ensure accountability** through auditing and traceability.
-

SAMPLE ANSWER

In a secure e-commerce platform, security begins with **authentication**, verifying user identity through secure logins or two-factor verification. Once verified, **authorisation** determines what the user can do — customers can view and purchase products, while administrators can manage inventory. To maintain **confidentiality**, sensitive information such as payment details is encrypted and only visible to authorised users. **Integrity** ensures that transactions are accurate and cannot be tampered with through validation checks and secure APIs. **Availability** is maintained through reliable servers, backups, and DDoS protection, ensuring customers can access the site at any time. Finally, **accountability** is upheld through detailed logging and monitoring of transactions, allowing administrators to trace activity and detect fraud. Together, these six principles establish a strong, layered security foundation for the e-commerce system.



APPLYING SECURITY FEATURES IN SOFTWARE DEVELOPMENT

SECURE-BY- DESIGN SOFTWARE DEVELOPMENT



Secure-by-Design is a software development approach where **security is embedded into every stage of the software development life cycle (SDLC)** — from requirements analysis and system design to coding, testing, deployment, and maintenance.



It focuses on **proactively preventing vulnerabilities** rather than reacting to security breaches after release. Developers design systems with security controls, checks, and principles built in from the start.



Goal:

To create software that is resilient, trustworthy, and compliant — making security a **core feature**, not an optional extra.

1. DATA PROTECTION

Definition

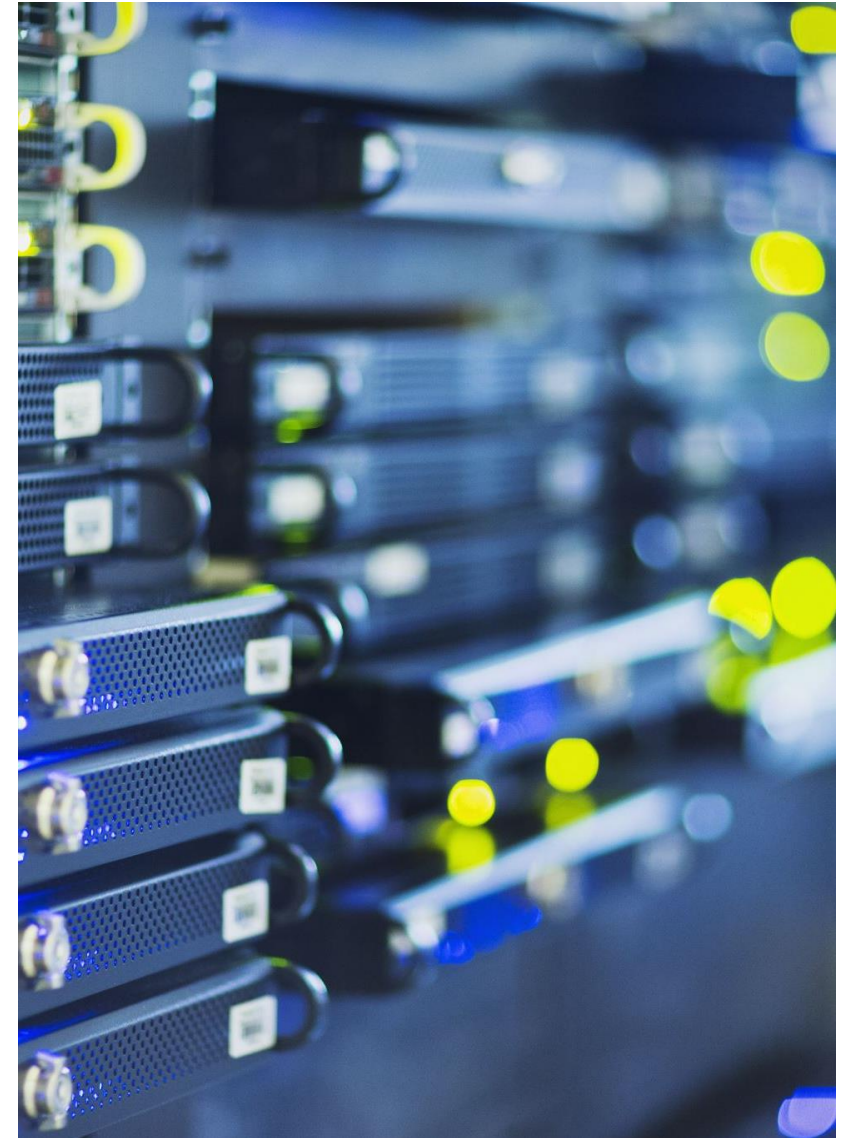
Data protection refers to the **safeguarding of information** from corruption, unauthorised access, or loss during storage, transmission, and processing.

Objectives

- **Confidentiality:** Only authorised people can access data.
- **Integrity:** Data remains accurate and consistent.
- **Availability:** Data is accessible when needed.

Example

- A school management system encrypts student records, uses password-protected access for staff, and validates data entry to protect against corruption.



TECHNIQUES

Technique	Description	Example
Encryption	Encodes data to prevent unauthorised access.	AES encryption for passwords.
Access Control	Restricts who can view or modify information.	Admin-only access to student medical records.
Backups & Recovery	Creates copies to prevent data loss.	Automated daily backups to cloud storage.
Integrity Checks	Uses hashes to detect tampering.	SHA-256 file verification.



2. SECURITY

Definition

- Security ensures that a system and its data are **protected against cyberattacks, misuse, and unauthorised access.**

Example

- An online learning portal uses encrypted communication (HTTPS), validates all input fields, and limits login attempts to prevent brute-force attacks.

CORE SECURITY PRACTICES

Practice	Description	Example
Authentication	Confirms user identity.	Username + password + 2FA.
Authorisation	Determines access rights.	Admins can modify, students can view.
Input Validation	Prevents malicious input.	Sanitising HTML forms to block XSS.
Error Handling	Avoids revealing system details.	Generic error messages.
Patch Management	Updates software to fix vulnerabilities.	Regular system updates.

3. PRIVACY

Definition

- Privacy focuses on **how personal data is collected, stored, used, and shared**, ensuring users have **control and transparency** over their information.

Key Techniques

- **Data Minimisation:** Collect only essential data.
- **User Consent:** Ask for permission before data collection.
- **Anonymisation:** Remove identifiable information.
- **Transparency:** Clear communication about data use.

Example

- A fitness tracking app anonymises user statistics for group comparisons and requires user consent before sharing location data.

PRIVACY BY DESIGN PRINCIPLES

Principle	Explanation	Example
Proactive not Reactive	Prevent privacy breaches before they happen.	Privacy risk assessment during design phase.
Embed Privacy into Design	Build privacy features directly into the system.	Anonymising user data before analysis.
Respect for User Privacy	Give users control and clarity.	Letting users opt in/out of data sharing.

4. REGULATORY COMPLIANCE

Definition

- Regulatory compliance means ensuring software meets **legal and ethical standards** for data handling, storage, and privacy.

Developer Responsibilities

- Securely store and transmit personal data.
- Obtain user consent before data collection.
- Provide users with access to their data.
- Maintain logs and audit trails for accountability.
- Implement breach detection and reporting processes.

Example

- An Australian online retailer complies with APPs by encrypting all customer data, obtaining explicit consent for marketing emails, and providing an option to delete user accounts.

RELEVANT LAWS AND STANDARDS

Regulation	Region	Focus
Privacy Act 1988 (Cth)	Australia	Protects personal information collected by organisations.
Australian Privacy Principles (APPs)	Australia	13 principles for handling and disclosing personal data.

REAL-WORLD EXAMPLE: ONLINE RETAIL PLATFORM

Security Feature	Implementation	Outcome
Data Protection	Customer details encrypted and backed up nightly.	Prevents data loss and theft.
Security	Multi-factor authentication and limited login attempts.	Prevents unauthorised access.
Privacy	Clear consent forms and anonymised analytics.	Builds user trust.
Regulatory Compliance	Follows Privacy Act 1988 and APPs.	Meets legal and ethical standards.

Concept	Description	Example
Secure by Design	Build security from the start of development.	Encryption, access control, privacy checks during design.
Data Protection	Safeguard data against loss or misuse.	Encryption, backups.
Security	Defend software from attacks.	Authentication, input validation.
Privacy	Respect user data rights and consent.	Data minimisation, transparency.
Regulatory Compliance	Follow laws and standards.	GDPR, APPs, PCI DSS.

SUMMARY TABLE

HSC READY SUMMARY

Secure-by-design software development means building security, privacy, and compliance into the system from the very beginning, rather than adding them later. This approach ensures data protection through encryption, access control, and regular backups, maintaining confidentiality, integrity, and availability of information. Security mechanisms such as authentication, authorisation, and input validation prevent unauthorised access and reduce vulnerabilities. Privacy is protected by collecting only essential data, gaining user consent, and anonymising personal information, while regulatory compliance ensures adherence to standards like the **Australian Privacy Principles (APPs)**, which govern how data is collected, stored, and disclosed. Together, these practices create software that is resilient, lawful, and trustworthy.

HSC READY EXAMPLE

A practical example is an **e-commerce platform** like Amazon or eBay. Customer data such as credit card details is encrypted during checkout (data protection) and transmitted over HTTPS (security). Users authenticate via secure login and two-factor verification, while authorisation controls ensure only employees with specific roles can access sensitive information. Privacy policies explain how customer data is used and provide options to delete accounts (privacy by design). The system complies with privacy legislation to ensure all transactions meet global data protection laws. This integration of data protection, security, privacy, and compliance demonstrates a secure-by-design approach .



CRYPTOGRAPHY AND SANDBOXING IN SECURITY BY DESIGN

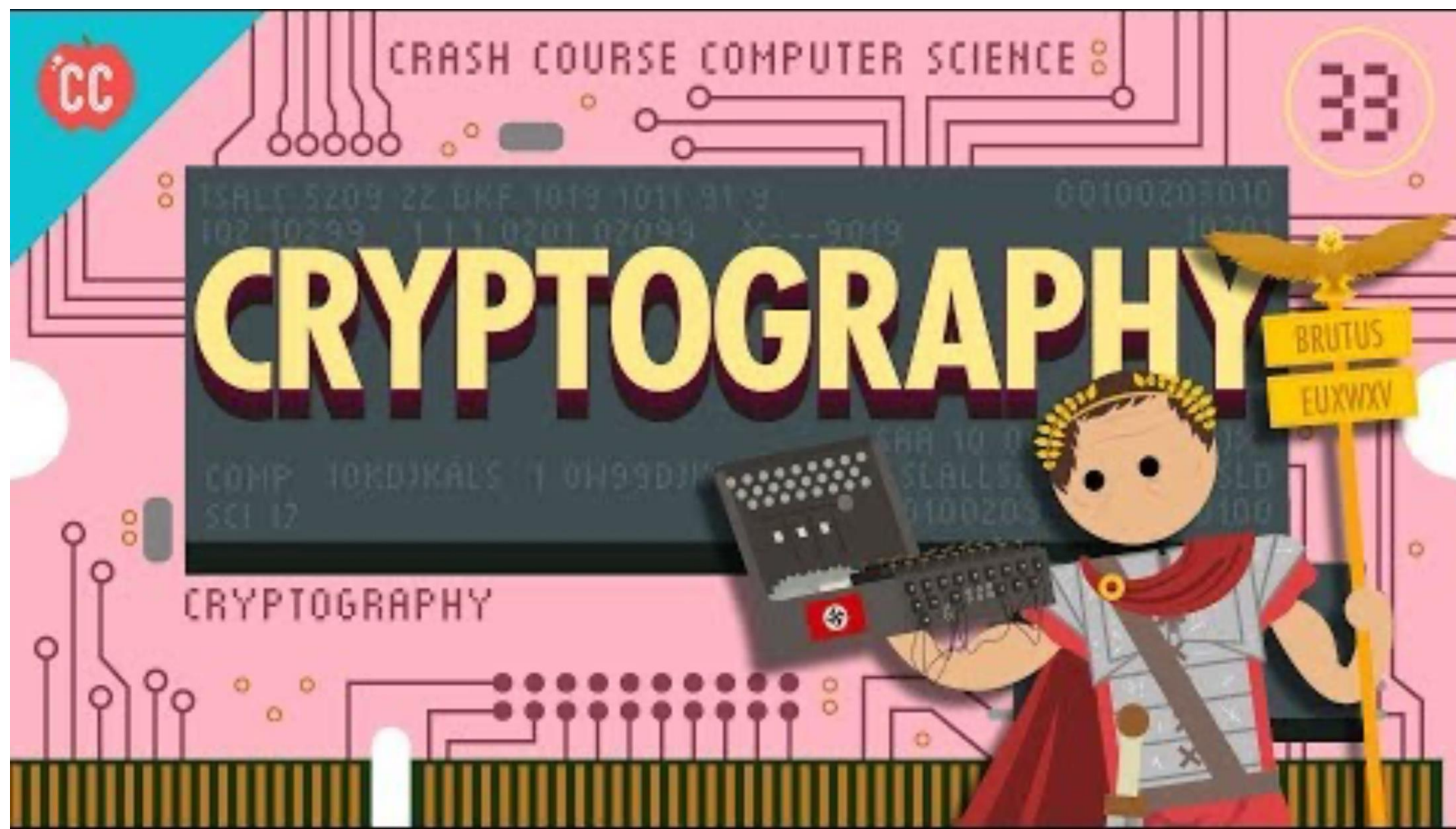
CRYPTOGRAPHY

Ensuring the **confidentiality** of data is now a basic requirement of computing. **Cryptography** underpins everything from sending secure messages to making online purchases. By the end of this module, you will appreciate how cryptography protects our digital activities and learn several foundational techniques.

Why Is Cryptography Important?

- **Protecting Data:** Prevents unauthorised parties from reading private information.
- **Verifying Authenticity:** Ensures a message or file really comes from who it claims to.
- **Maintaining Integrity:** Detects if any data is tampered with during transmission or storage.

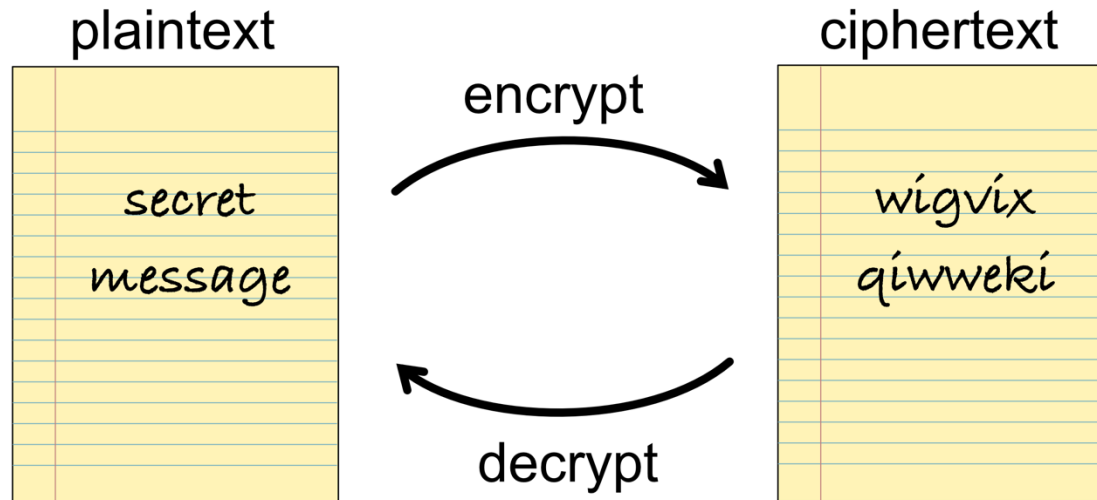




CIPHERS

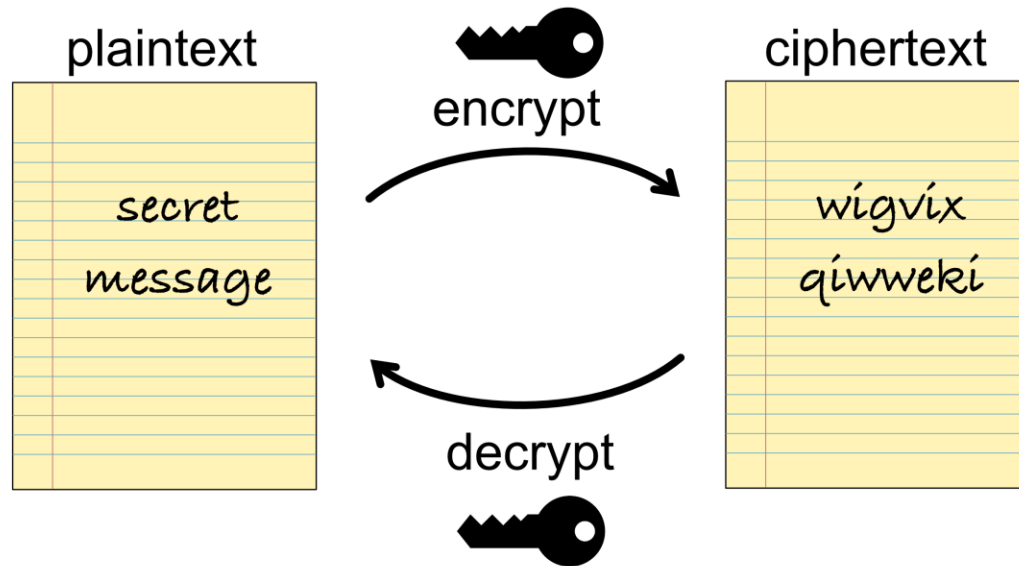
- Ciphers are a fundamental part of cryptography, serving as the method used to transform readable information, called **plaintext**, into obscured data, called **ciphertext**. By applying specific steps, usually mathematical operations, ciphers convert potentially sensitive information into a form that can only be understood by those who have knowledge on how to reverse it.
 - The primary purpose of a cipher is to secure information against unauthorised access. Whether we are protecting online transactions, guarding personal communications, or shielding government intelligence, ciphers ensure that data remains private.
 - The process of applying a cipher to plain text is called **encryption**. The reverse process, converting cipher text to plain text, is called **decryption**. The decryption process requires knowledge of the cipher and often additional information such as a **key**.
-

USING CIPHERS



1. **Write your data in plaintext.** It could be a message, a document, or any data that needs protection.
2. **Encrypt the plain text.** Using a specific cipher and an associated key, the plain text is scrambled.
3. **Store or send the ciphertext.**
4. **Decrypt the cipher text.** The recipient applies the method and/or key used to encrypt the data. This reverses the encryption process and retrieves the original plain text.

SYMMETRIC-KEY CRYPTOGRAPHY



- In symmetric-key cryptography the **same key** is used to both **encrypt** the plaintext to ciphertext and **decrypt** the ciphertext to plaintext.

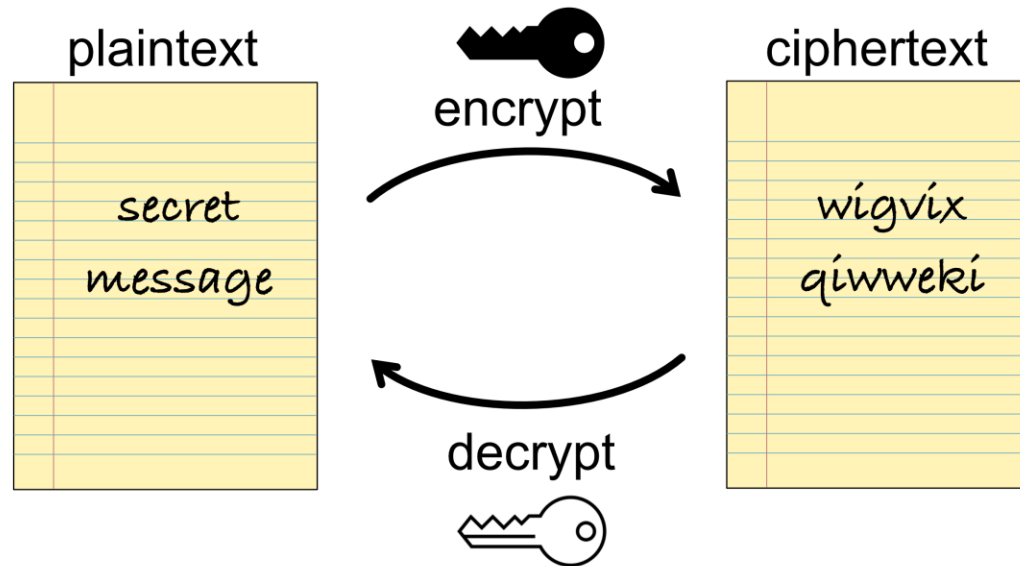
In Practice

- Simple ciphers like a substitution cipher are vulnerable to brute force attacks or frequency analysis and can be broken. Modern encryption algorithms incorporate several clever techniques to create encryption that is resistant to these kinds of attacks.
- Symmetric-key encryption algorithms achieve a high level of security in two ways:
 - **Confusion**, which is decorrelating the input plain text from the output cipher text
 - **Diffusion**, which is hidings the plain text by spreading it over a greater number of characters

DISADVANTAGES

The main disadvantage of symmetric-key cryptography is the **both parties need to know what the key is**. For example, if Alice wanted to send a secret message to Stevie using a symmetric cipher, both Alice and Stevie need to know what the key is. But say Alice and Stevie are in two different cities, how do they share the secret key? They can't encrypt anything yet because they need to share a key to encrypt, so they would have to send the key securely somehow in plaintext, which runs the risk that someone else sees the key. The most secure way to share the key would be to meet in person to share the key, but this could be quite impractical, especially if they wanted to periodically change their secret key. We'll see later that you can get around this using **asymmetric cryptography**.

ASYMMETRIC-KEY CRYPTOGRAPHY



- In asymmetric cryptography (public-key cryptography) there are **two keys**, which form a **key-pair**. One key is used to **encrypt** the plaintext to ciphertext and one key is used to **decrypt** the ciphertext to plaintext. Note that **you cannot encrypt and decrypt using the same key**. If you encrypt with one key you can only decrypt it with the other key.
- Asymmetric cryptography can also be used to prove identity. For example, if Alice wants to publish something and guarantee that she is the author, she can **encrypt** it with her **private key**. Now everyone else in the world will be able to **decrypt** her work using the **public key**, but in doing so, they will know that Alice must have been the person to have originally encrypt it because she is the only person who knows her private key.

COMBINING ASYMMETRIC AND SYMMETRIC ENCRYPTION

- Because of the issues with asymmetric encryption the usual protocol is to:
 - Use an asymmetric encryption algorithm to establish a symmetric key between two parties
 - Encrypt and decrypt the bulk of the data using the symmetric key

HASHING AND DIGITAL SIGNATURES

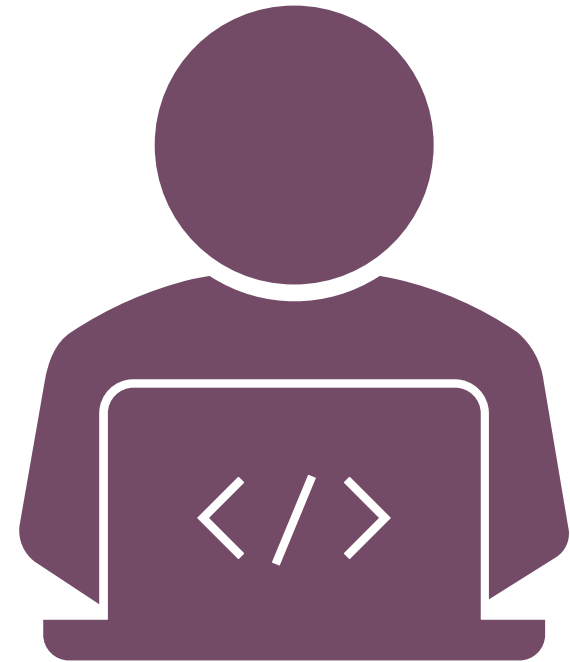
- A **hash function** converts data into a fixed-length value, called a hash, which acts as a digital fingerprint for the original data. Even a small change to the original data will result in a completely different hash value, which makes it easy to detect if the data has been tampered with.
- When a user creates a password, the password is hashed before being stored in the database. If an attacker gains access to the database, they will only see the hashed passwords, not the original ones.
 1. Plain password: "mypassword"
 2. Hashed password: "5f4dcc3b5aa765d61d8327deb882cf99"
- If the user later tries to log in, the system hashes the password they provide and compares it to the stored hash. If they match, the login is successful.

CRYPTOGRAPHY SUMMARY

Cryptography is the science of securing data through encoding techniques. It ensures **confidentiality**, **integrity**, and **authentication** in software systems. By encrypting sensitive data during storage and transmission, cryptography prevents unauthorised access even if data is intercepted. Techniques such as **symmetric encryption** (e.g., AES) and **asymmetric encryption** (e.g., RSA) protect information, while hashing algorithms (e.g., SHA-256) verify data integrity. Digital signatures and certificates authenticate users and systems, reducing risks of impersonation. In exams, emphasise that cryptography is embedded in design to secure communication channels, databases, and APIs from the start.

SANDBOXING

- **Sandboxing** is the practice of running code in a restricted environment where it has limited access to the system and resources. This helps prevent malicious code or untrusted applications from harming the system.
- **Sandboxing** could be used to run user-submitted scripts or bots in a controlled environment, preventing them from accessing or manipulating system files, data, or even interacting with other parts of the application. By creating sandboxed environments for development and testing, you can safely test new features, including group management and financial transactions, without risking damage to the production system.





The Benefits Of Sandboxing



Create and deploy environments



Gain access to advanced networking and support



Prepare for future attacks



Enhance collaboration



Save your company money

BENEFITS OF SANDBOXING

SANDBOXING

Sandboxing isolates applications or processes in a controlled environment, preventing them from affecting the rest of the system. This technique is crucial for testing untrusted code, running third-party applications, or executing scripts safely. By restricting access to system resources, sandboxing minimises the impact of malware or vulnerabilities. For example, web browsers use sandboxes to run plugins without compromising the operating system. In an exam, explain that sandboxing supports security by design by containing potential threats and enforcing the principle of least privilege.

CONTRIBUTION TO SECURITY BY DESIGN

Both cryptography and sandboxing are proactive measures. Cryptography secures data and communication from the outset, while sandboxing limits the damage from malicious or faulty code. Together, they create layered defenses that align with security by design principles, ensuring software is resilient against attacks before deployment.

PRIVACY BY DESIGN IN SOFTWARE DEVELOPMENT

WHAT IS PRIVACY BY DESIGN?

- **Privacy by Design (PbD)** is a software development approach where **privacy is built into the system from the beginning** — it's not added later as an optional feature.
- It ensures that **users' personal data is automatically protected** by default, through thoughtful system architecture, responsible coding practices, and transparent user policies.
- In short: *Privacy by Design means anticipating and preventing privacy risks before they happen.*



WHY IT MATTERS



Builds trust between users and developers.



Prevents data breaches and legal issues.



Supports compliance with Privacy Act 1988 (Cth) & APPs.



Promotes ethical software development.

PRINCIPLE 1 – PROACTIVE, NOT REACTIVE

Meaning:

- Developers must **anticipate privacy risks early** in the design stage and build systems that prevent breaches before they occur.
- Rather than reacting to problems (e.g., after a data leak), teams identify possible risks and address them during development.

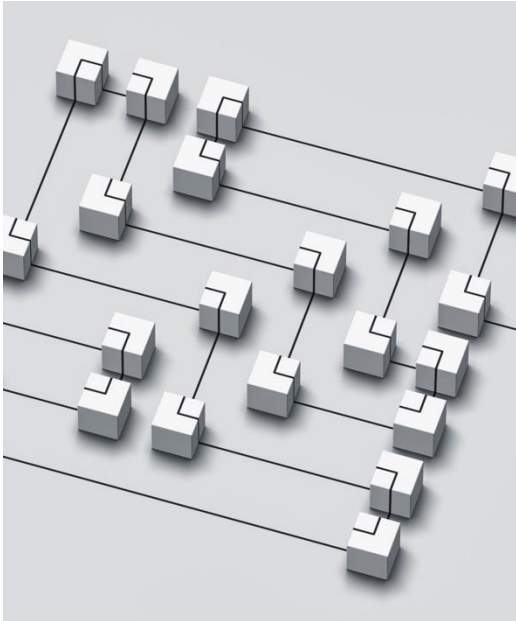
How It's Done:

- Use secure coding and storage methods from the start.
- Establish a **breach response plan** before deployment.
- Include privacy in user stories and requirements documentation.

Example:

- Before launching a **school management app**, developers identify that parent contact information could be misused. They proactively design **role-based access control** so teachers can see student data but not other families' private information.

PRINCIPLE 2 – EMBED PRIVACY INTO DESIGN



Meaning:

- Privacy and data protection are **integrated into the software architecture**, not tacked on later. It becomes a **core design feature**, similar to performance or usability.

How It's Done:

- **Data minimisation:** Collect only essential personal information.
- **Encryption:** Protect data at rest and in transit.
- **Secure defaults:** Private by default, share only if the user opts in.
- **Access control:** Limit who can access sensitive data.

Example:

- A **health tracking app** automatically encrypts user data, anonymises statistics for research, and allows users to decide what health information they share. These privacy protections are part of the design, not an afterthought.

PRINCIPLE 3 – RESPECT FOR USER PRIVACY

Meaning:

- Respecting user privacy means being **transparent**, **ethical**, and **user-centred**. Users should know what data is collected, why it's needed, and how they can control it.

How It's Done:

- Obtain **informed consent** before collecting personal data.
- Provide **clear privacy policies** in plain language.
- Allow users to **access, modify, or delete** their information.
- Avoid selling or sharing data without explicit permission.

Example:

- An **e-commerce website** clearly explains how user data will be used (e.g., for delivery, not advertising). Customers can review and delete their purchase history. This openness respects user privacy and builds trust.

REAL-WORLD EXAMPLE – APPLE IOS



Apps must request permission before accessing location/contacts.



On-device Face ID/Touch ID processing.



?? Privacy 'nutrition labels' show data usage.



Apple demonstrates proactive, embedded, and user-respecting privacy design.

SUMMARY TABLE

Proactive, not reactive: Prevent issues early.

Embed privacy into design: Build protections into architecture.

Respect user privacy: Give control and transparency.

Example: Risk audit, encryption, user data deletion.

EXAM READY SUMMARY

The **Privacy by Design** approach ensures that privacy is integrated into software systems from the beginning, rather than being added after development. It is a **proactive, not reactive** strategy — developers anticipate privacy risks early and prevent them before they happen. Privacy is **embedded into design**, with protections like encryption, data minimisation, and access control built into the architecture. This ensures **users' privacy** is protected automatically, not through optional settings.

For example, a health tracking app might encrypt fitness data, allow users to choose what metrics to share, and provide a “delete my data” feature. By embedding privacy into software from the start, developers comply with laws such as the **Australian Privacy Principles** and build systems that earn user trust and demonstrate ethical responsibility.

TESTING AND EVALUATING SOFTWARE SECURITY & RESILIENCE



OVERVIEW

- Secure software development requires **constant testing, evaluation, and improvement**. Security isn't only about preventing attacks — it's about **ensuring resilience**: the system's ability to **withstand, detect, respond to, and recover from threats**.
 - Developers use a combination of **code-level security strategies** (like SAST, DAST, and code reviews) and **system-level evaluations** (like vulnerability assessments, hardening, and disaster recovery) to create software that remains **trustworthy and operational under pressure**.
 - The goal: build systems that are not just functional — but secure, reliable, and resilient.
-

DETERMINING VULNERABILITIES

Vulnerability	Description	Example
Injection	Malicious code inserted into input fields.	SQL or XSS injection.
Broken Authentication	Flaws in login or session management.	Reusing session IDs.
Insecure APIs	Exposed endpoints or weak encryption.	Public API without tokens.
Unpatched Software	Outdated libraries or OS versions.	Using old frameworks.

- **Definition:**
A **vulnerability** is a flaw or weakness that attackers can exploit to compromise a system or its data.
- **How to Identify:**
Developers use several tools and methods to detect vulnerabilities **early and continuously**.

CODE SECURITY STRATEGIES – 1. CODE REVIEW

Code Review

- A **manual inspection** of code by developers to find security related code errors, unsafe practices, or missed validation. Common issues found during code review include missing input validation, hardcoded passwords, and poor error handling.
 - Detects errors that automated tools may miss.
 - Promotes collaboration and shared accountability.
 - Helps maintain code quality and security consistency.
 - **Example:** During a school voting app project, a code review finds that candidate names are not validated, which could allow malicious code injection.
-

CODE SECURITY STRATEGIES – 2. STATIC APPLICATION SECURITY TESTING (SAST)

- **SAST** analyses source code **before the program runs**. It scans for vulnerabilities in syntax, insecure libraries, and unsafe coding patterns.
 - Fast and automated.
 - Ideal during development (build pipeline).
 - Detects issues like buffer overflow, injection flaws, or hardcoded passwords.
- **Example:**

Before running a homework submission app, a SAST tool flags a line where user input is directly concatenated into an SQL query, which could allow a hacker to delete all homework records.

CODE SECURITY STRATEGIES – 3. DYNAMIC APPLICATION SECURITY TESTING (DAST)

- **DAST** tests the application **while it is running**, simulating attacks from an external perspective.
 - Identifies vulnerabilities that appear during execution.
 - Detects issues like cross-site scripting (XSS), broken authentication, or insecure session handling.
 - Complements SAST — tests what happens in the real world.
- **Example:**

A school event registration website is tested using DAST, which sends `<script>alert('XSS')</script>` to input fields to check for XSS vulnerabilities.

COMPARE AND CONTRAST SAST AND DAST

Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) are two key strategies for identifying vulnerabilities in software, but they differ in approach and timing. **SAST** analyses the source code or binaries without executing the program, making it ideal for detecting issues early in the development phase, such as insecure coding practices or SQL injection risks. For example, a SAST tool might flag unsanitised user input in a student portal before the app is deployed. In contrast, **DAST** tests the application while it is running, simulating real-world attacks to uncover runtime vulnerabilities like broken authentication or exposed APIs. For instance, a DAST scan on a live e-commerce site could reveal that the login page is susceptible to brute-force attacks.

COMPARE AND CONTRAST SAST AND DAST PART 2

- **Key differences:**
 - **Timing:** SAST is used during development; DAST is used on running applications.
 - **Scope:** SAST focuses on code-level issues; DAST identifies runtime and environment-related flaws.
 - **Access:** SAST requires access to source code; DAST does not.
 - **Both** methods complement each other—SAST prevents vulnerabilities early, while DAST ensures security in real-world conditions.
-

CODE SECURITY STRATEGIES – 4. VULNERABILITY ASSESSMENT

- A **systematic review** of all potential security weaknesses — combining automated scanning and manual analysis.
 - Prioritises vulnerabilities by severity (low, medium, high).
 - Produces reports with mitigation strategies.
 - Should be repeated regularly to maintain security.
- **Example:**

A school library system uses an outdated web framework, and a vulnerability assessment tool alerts that this version has a known flaw allowing hackers to bypass login.

CODE SECURITY STRATEGIES – 5. PENETRATION TESTING (PEN TESTING)

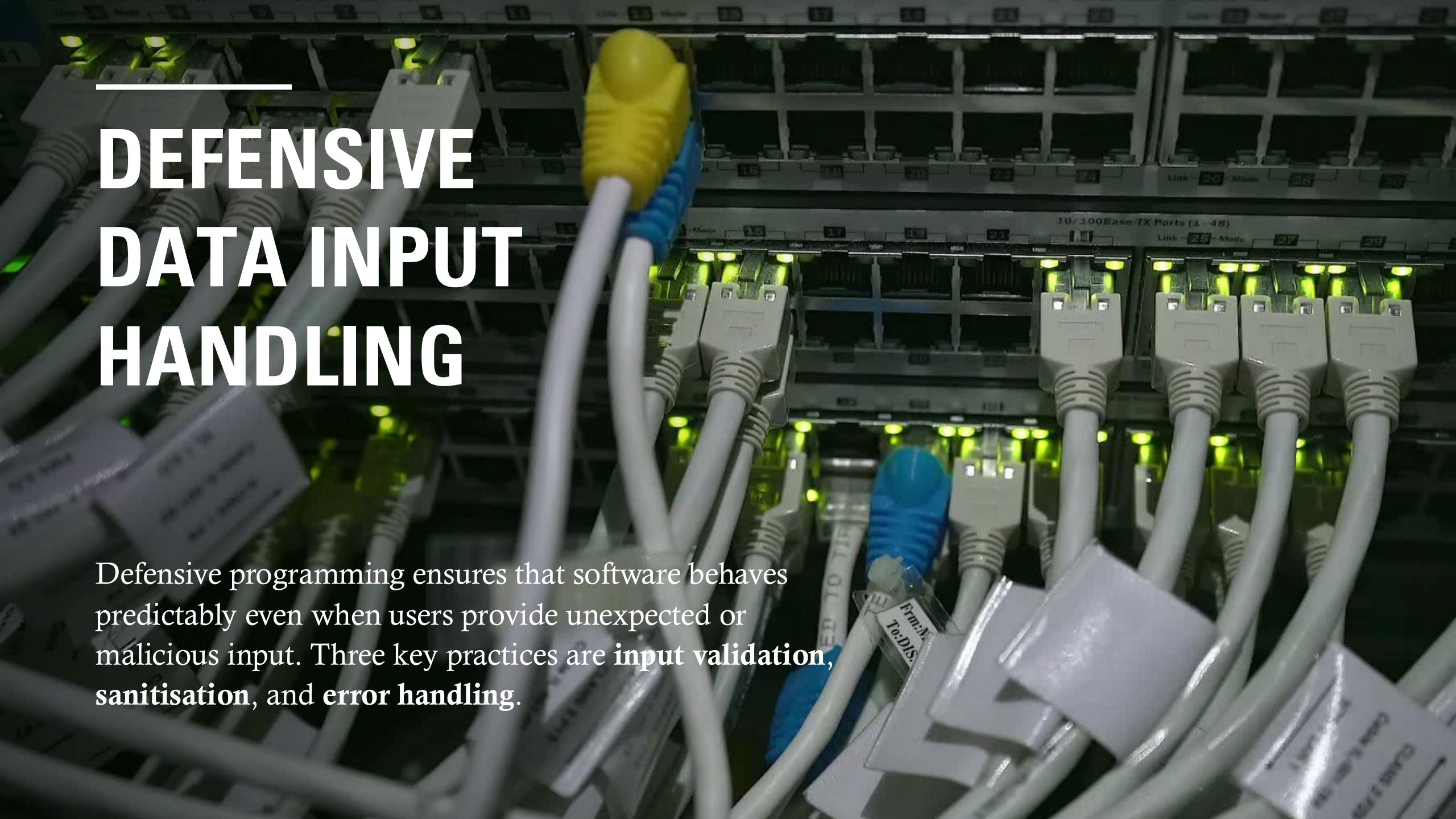
- An **ethical hacking process** where testers simulate cyberattacks to exploit vulnerabilities — testing the system's defences and incident response.
 - Goal: Exploit weaknesses to show potential impact, and provides actionable insights for fixing issues.
 - Types:
 - Black-box (no prior knowledge).
 - White-box (full knowledge).
 - **Example:**

Your school hires a cybersecurity expert to test the student portal. They manage to log in as a teacher without credentials by exploiting a weak session management flaw. This shows why penetration testing is critical before going live.
-

SECURITY TESTING APPROACHES

Phase	Security by Design Processes
Requirements definition	• Gather specific security and privacy requirements • Vulnerability assessment
Determining specifications	• Explicit security and privacy specifications • Risk assessment
Design	• Threat modelling • Security design review • Security tests included in test designs
Development	• Code reviews • Static application security testing

Integration	• Risk assessment • Code reviews • Dynamic application security testing • Grey-box penetration testing
Testing and debugging	• Code reviews • Static application security testing • Dynamic application security testing • Penetration testing
Installation	• Penetration testing • Vulnerability assessment
Maintenance	• Log monitoring & reporting • Vulnerability assessment

A close-up photograph of a network switch or patch panel. Numerous white Ethernet cables are plugged into the ports, with some featuring yellow and blue RJ45 connectors. Several green status LEDs are illuminated, indicating active connections. The background is dark, and the overall scene is dimly lit, focusing on the cables and lights of the network hardware.

DEFENSIVE DATA INPUT HANDLING

Defensive programming ensures that software behaves predictably even when users provide unexpected or malicious input. Three key practices are **input validation**, **sanitisation**, and **error handling**.

INPUT VALIDATION

- **Definition:** The process of checking that all incoming data meets expected rules before it is processed.
- **Purpose:** Prevents invalid or harmful data from entering the system.
- **Key checks:**
 - **Type:** Ensure data is the correct type (e.g., number, string).
 - **Format:** Match patterns (e.g., email addresses, dates).
 - **Range:** Confirm values fall within acceptable limits.
 - **Length:** Restrict size to prevent buffer overflow or resource abuse.
- **Best practice:** Use **allow-lists** (what is permitted) rather than block-lists.

SANITISATION



- **Definition:** Cleaning or transforming data to remove harmful elements before storage or display.
- **Purpose:** Prevents attacks such as cross-site scripting (XSS) or SQL injection.
- **Approach:**
 - Normalise input (trim spaces, standardise case).
 - Encode output for its context (HTML, URL, JavaScript).
 - Use parameterised queries for databases instead of string concatenation.

ERROR HANDLING



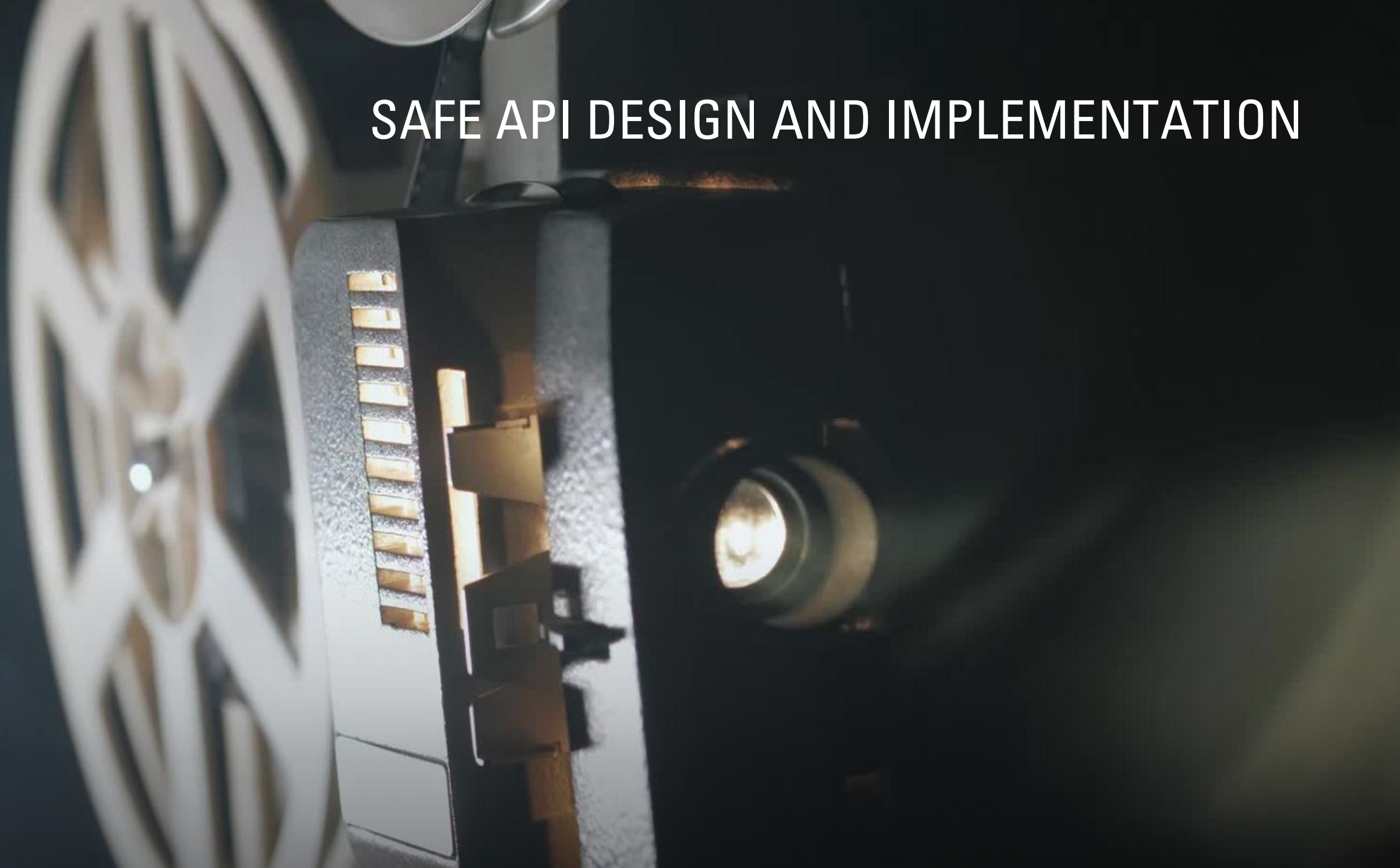
- **Definition:** Managing unexpected conditions gracefully without exposing sensitive information.
- **Purpose:** Maintains security and user experience when errors occur.
- **Principles:**
 - Provide **clear, helpful messages** to users without revealing system details.
 - Log detailed error information securely for developers.
 - Avoid exposing stack traces or database queries in user-facing messages.
 - Fail safely—if validation fails, reject the input and stop processing.

WORKFLOW SUMMARY

1. **Receive input** → normalise.
2. **Validate** type, format, range, length.
3. **Sanitise** before storage or display.
4. **Process safely** using secure APIs and prepared statements.
5. **Handle errors** with user-friendly messages and secure logging.



SAFE API DESIGN AND IMPLEMENTATION





APPLICATION PROGRAMMING INTERFACE

- Application Programming Interface (API) is an interface that allows an application or website to plug into another program or website.
- APIs (Application Programming Interfaces) allow different software components to communicate, but poorly designed APIs can introduce serious security risks. A safe API must enforce **data integrity**, **authentication**, and **secure communication**.

**MORE API WILL BE COVERED IN
THE NEXT UNIT**

**EFFICIENT CODE
EXECUTION
FOR THE USER**

EFFICIENT CODE EXECUTION FOR THE USER

- Efficient execution means software runs smoothly, uses resources wisely, and provides a responsive experience. Three key considerations are **memory management**, **session management**, and **exception management**.
-

MEMORY MANAGEMENT

Definition: Controlling how a program allocates, uses, and releases memory during execution.

- **Why it matters:** Poor memory handling can lead to slow performance, crashes, or security vulnerabilities.
 - **Best practices:**
 - Avoid unnecessary data duplication.
 - Release unused resources promptly (e.g., close files, free objects).
 - Use efficient data structures suited to the task.
 - Prevent memory leaks by ensuring all allocated memory is freed when no longer needed.
 - If a program uses too much memory or does not release it (*memory leak*), it limits the memory available to other programs.
-

KEY TERMS

Term	Meaning
Memory Allocation	Assigning space in memory for variables and data.
Deallocation	Releasing memory when it's no longer needed.
Memory Leak	When memory is not freed properly, causing wasted resources.
Garbage Collection	Automatic memory cleanup by some programming languages (e.g. Python, Java).

EXAM READY

- Memory management affects how quickly and smoothly a program runs by controlling how memory (RAM) is used and released during execution.
If a program uses memory efficiently, it can load data quickly and respond faster to user input.
 - For example, a **game that properly clears unused objects** (like old player positions or level data) will continue running smoothly without lag. However, if the program fails to release this memory — known as a **memory leak** — the game may start to slow down, freeze, or crash after running for a while.
 - Efficient memory management therefore ensures **fast performance and a positive user experience**, while poor management leads to **slow execution and instability**.
-

SESSION MANAGEMENT

- **Definition: Session management** is the process of tracking and maintaining a user's interactions with a software system or website over time — often using cookies, tokens, or session IDs — so the system can “remember” the user's state and data between actions.
- **Why it matters:** Improper session handling can cause performance issues and security risks like session hijacking.
- **Best practices:**
 - Use short, reasonable session timeouts to free resources.
 - Store minimal session data to reduce memory usage. Session management allows only **necessary parts of code** to run when the user interacts again.
 - Regenerate session IDs after login to prevent fixation attacks.
 - Encrypt session tokens and transmit them over secure channels (HTTPS).

KEY TERMS

Term	Meaning
Session	A period during which a user interacts with a system.
Cookie / Token	Data stored to identify or authenticate a user between requests.
Session Timeout	Automatic end of a session after inactivity for security and resource control.
Cache	Temporarily stored data that speeds up access.

EXAM READY

Session management improves efficiency by remembering a user's state so that only necessary code runs when they return or move between pages. For example, a **shopping website** that keeps a user logged in and remembers items in their cart avoids reloading all user data each time they click a new page. However, if sessions are not managed properly or kept active too long, the system's memory and cache can become overloaded, slowing performance for all users. Efficient session management therefore keeps programs **fast, responsive, and secure**.

EXCEPTION MANAGEMENT

- **Definition: Exception management** is the process of detecting, handling, and responding to errors or unexpected events that occur during program execution, so the software can continue running without crashing.
 - **Why it matters:** Poor error handling can degrade user experience and reveal internal system information.
 - **Best practices:**
 - Catch and handle exceptions at appropriate levels.
 - Provide clear, user-friendly error messages without technical details.
 - Log detailed error information securely for developers.
 - Fail safely—avoid leaving the system in an inconsistent state.
-

KEY TERMS

Term

Meaning

Exception

An error or unexpected event that disrupts normal program execution.

Try-Except Block

Code structure used to handle exceptions safely.

Error Message

Information displayed to the user when something goes wrong.

Debugging

The process of finding and fixing the cause of exceptions.

EXAM READY

Exception management allows a program to continue running when an error occurs, improving its stability and efficiency.

For example, in a **Python login program**, if a user enters text instead of a number for their PIN, the program can use a try–except block to catch the `ValueError` and prompt the user to try again instead of crashing.

This prevents interruption, maintains system stability, and improves the overall user experience.

Efficient exception management therefore ensures programs **handle errors gracefully and run reliably**.

VULNERABILITIES IN USER ACTION CONTROLS



PRAC- STUDENTS ARE REQUIRED TO KNOW THESE VULNERABILITIES AND THEIR FIXES

- SQL Injection: <https://www.hacksplaining.com/lessons/sql-injection/start>
- Cross Site Scripting: <https://www.hacksplaining.com/lessons/xss-stored/start>
- Cross Site Forgery Request: <https://www.hacksplaining.com/lessons/csrf/start>
- Invalid forwarding and redirecting: <https://www.hacksplaining.com/lessons/open-redirects/start>
- Memory management: <https://www.hacksplaining.com/lessons/information-leakage/start>
- Session mismanagement: <https://www.hacksplaining.com/lessons/session-fixation/start>
- Broken Authentication: <https://www.hacksplaining.com/lessons/password-mismanagement/start>
- Race conditions: [Foundations of Cybersecurity 2-6: Race Conditions](#)



XSS-CROSS SITE SCRIPTING

- **What is cross-site scripting (XSS)?**
- Cross-site scripting (also known as XSS) is a web security vulnerability that allows an attacker to compromise the interactions that users have with a vulnerable application. Cross-site scripting vulnerabilities normally allow an attacker to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data. If the victim user has privileged access within the application, then the attacker might be able to gain full control over all of the application's functionality and data.
- **How does XSS work?**
- Cross-site scripting works by manipulating a vulnerable web site so that it returns malicious JavaScript to users. When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.

SQL INJECTION

SQL injection is a type of cyber attack that exploits vulnerabilities in web applications by manipulating SQL queries through user input fields. It occurs when an application does not properly validate or sanitise user input, allowing attackers to insert malicious SQL code into a query. This can result in unauthorised access to databases, data breaches, or even deletion of important information. For instance, if a login form directly inserts user input into a SQL statement without checks, an attacker could enter ' OR '1'='1 to bypass authentication. Preventing SQL injection involves using prepared statements, parameterised queries, and thorough input validation to ensure user data cannot alter the structure of SQL commands. Understanding and mitigating SQL injection is essential for developing secure software systems.

XSS (**Cross-Site Scripting**) and CSRF (**Cross-Site Request Forgery**) are both web security vulnerabilities, but they exploit different weaknesses and have distinct attack mechanisms.

Feature	XSS (Cross-Site Scripting)	CSRF (Cross-Site Request Forgery)
Purpose	Injects malicious scripts into a trusted website to execute on a victim's browser.	Tricks a user into unknowingly making unwanted actions on a trusted website.
Attack Method	Attacker injects JavaScript or HTML into a web page, which executes in the user's browser.	Attacker exploits an authenticated user session by sending unauthorized requests.
Exploits	Poor input validation, allowing attackers to inject scripts into web pages.	Lack of proper request validation (e.g., missing CSRF tokens).
Requires User Interaction?	Yes, the victim must visit a compromised or malicious web page.	Yes, the victim must be tricked into clicking a link or loading a webpage with a malicious request.
Effect on Users	Can steal cookies, session tokens, or credentials; deface web pages; spread malware.	Performs actions on behalf of an authenticated user, such as making purchases or changing account details.
Mitigation	- Input validation and output encoding	

Key Takeaways

- **XSS targets users** by injecting malicious scripts that execute in their browsers.
 - **CSRF targets actions** by tricking an authenticated user into making unintended requests.
 - Both can be mitigated with proper security measures like **input validation, CSRF tokens, and secure cookie settings**.
-

RACE CONDITION

- Race conditions refer to a situation where two or more database operations occur in parallel, and their order of execution affects the system's state or outcome.
- This scenario is particularly problematic in environments where multiple processes or users access and modify the same data simultaneously. Without proper handling, race conditions can lead to data integrity issues.
- Database systems use locking mechanisms and transaction isolation levels to manage concurrent access and prevent race conditions. Locks prevent multiple transactions from modifying the same data simultaneously, while transaction isolation levels define the degree to which a transaction must be isolated from modifications made by other transactions.





SECURE CODE FOR PREVENTING FILE ATTACKS & SIDE- CHANNEL ATTACKS

1. UNDERSTANDING FILE ATTACKS

- File attacks happen when attackers abuse the way a program handles files—such as file names, file paths, or uploaded files—to gain access to private information or damage the system.
- **What can go wrong?**
 - Attackers upload harmful files disguised as harmless files.
 - They trick the program into opening system files instead of the intended directory.
 - They overwrite important data by using sneaky file names.
 - They use symbolic links (fake pointers) to redirect the program to sensitive files.



HOW TO PREVENT FILE ATTACKS

- Good secure code will:
 - **Sanitise file names** so users cannot include `../` to escape directories.
 - **Restrict where files can be stored**, usually in a dedicated safe folder.
 - **Validate file types** using MIME type checks—not just extensions.
 - **Use least-privilege permissions** so files cannot be executed or overwritten unexpectedly.
 - **Perform atomic file writes** (write to a temp file, then rename it safely).
- **Relatable example**
- A school website lets students upload assignment files. A student uploads a malicious `.exe` file but renames it `assignment.pdf.exe`. The system trusts the file name and runs the file when a teacher views it. Secure file validation stops this from happening.

FILE ATTACKS SUMMARY

- File attacks occur when an attacker manipulates how software handles files—such as file paths, names, formats or permissions—to gain unauthorised access, upload malicious files, or overwrite important system resources. These attacks commonly exploit weak input validation, unsafe file storage locations, unrestricted uploads, or insecure directory permissions. To prevent file attacks, developers must sanitise file names, restrict directory traversal, validate file types using MIME detection, apply least-privilege permissions, and store user-generated content outside publicly accessible locations. Secure coding practices such as parameterised file operations and atomic file writes also ensure that attackers cannot trick the system into reading or modifying unintended files.
 - **Relatable Example**
 - A student assignment upload portal allows users to upload any file, and the system stores them directly in a public folder. A malicious student uploads a script disguised as a .pdf file, which the server automatically executes when a teacher views it, giving the attacker access to sensitive files on the school server.
-

2. SIDE-CHANNEL ATTACKS

- Side-channel attacks don't target the code directly—they target the *physical behaviour* of the computer. This includes things like timing, CPU usage, power consumption, and even sound or electromagnetic signals.
- **What side-channel attacks look like**
 - **Timing attacks:** measuring how long a password check takes.
 - **Cache attacks:** observing how the CPU stores temporary data.
 - **Power analysis:** reading power fluctuations to guess secret keys.
 - **Acoustic/EM attacks:** listening to keystrokes or reading signals.
- **Why they work**
- Even secure code may accidentally “leak” information about how it runs. Attackers use these clues to guess private data.



HOW TO PREVENT SIDE- CHANNEL ATTACKS

- Use **constant-time algorithms** when comparing passwords or keys.
 - Avoid branching decisions based on secret information.
 - Use established cryptographic libraries (they already include protections).
 - Add random delays (noise) when appropriate.
 - For hardware systems: shield the device, filter power signals, isolate memory.
- **Relatable example**
 - A login system checks a password one character at a time. If the first few characters are correct, the response is slightly faster. A hacker records the timing and figures out the password character by character.
-

SIDE-CHANNEL ATTACKS SUMMARY

- Side-channel attacks exploit indirect information that a system leaks through timing, power usage, CPU cache behaviour or electromagnetic signals rather than attacking the code directly. These attacks allow an attacker to infer secrets—such as passwords or cryptographic keys—based on how long operations take or how hardware responds when processing sensitive data. Preventing side-channel attacks involves using constant-time algorithms, avoiding branching logic on secret values, applying noise or randomised delays when appropriate, and relying on secure, professionally designed cryptographic libraries that already include side-channel resistance. In hardware-based environments, physical protections such as shielding and power-filtering may also be required.
 - **Relatable Example**
 - A web application compares a user's password character-by-character, responding slightly faster when the first few characters are correct. A hacker measures the difference in response time using automated tools and guesses the password one character at a time until they gain full account access.
-

3.PROTECTING USER FILE VULNERABILITIES

- When programs allow users to upload or store files, these files can become a major attack point.
- **Why user file handling is risky**
 - Users might upload harmful files (malware).
 - Attackers can overfill storage to crash the system (storage exhaustion).
 - Unsafe file names can overwrite important system files.
 - User files might be stored where anyone can access them.



HOW TO SECURE USER FILE HANDLING



- Developers should:
 - Only allow file uploads to safe, isolated folders.
 - Reject files with risky characters (`../`, `\`, `:`).
 - Check both the file extension AND the internal file type.
 - Set strict file permissions (read-only when possible).
 - Never trust user-provided metadata.
 - **Relatable example**
 - A student uploads a file named `../ ../ ../teachers/marks.xlsx` during a profile photo upload. Without path sanitisation, the system tries to write to the teachers' marks file and overwrites it—causing data loss.
-

PROTECTING USER FILE VULNERABILITIES SUMMARY

To protect user files from attacks, secure code must ensure that uploaded or stored files cannot be used to access the wider file system, execute malicious code, or overwrite sensitive data. Secure development practices include sanitising file paths, using safe directory joins, blocking dangerous characters (such as `../` traversal sequences), enforcing strict file permissions, and storing uploads in non-public, segregated directories. Developers should also perform server-side file type inspection, implement quotas to prevent storage exhaustion attacks, and avoid relying on user-provided file metadata. These measures ensure that user files remain isolated and cannot be used as an entry point for privilege escalation or system compromise.

Relatable Example

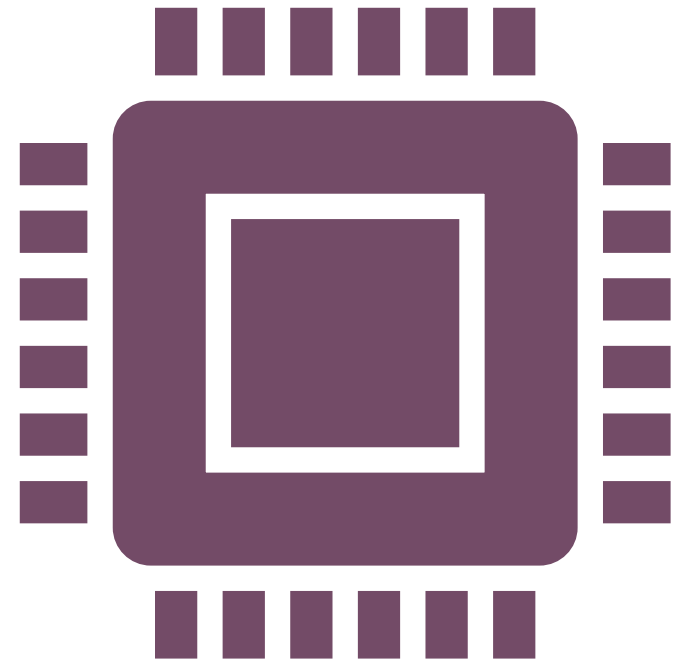
A school photo upload form stores images in a teacher-only area. Because the system fails to restrict file path access, a student uploads a file named `../ ../ ../marks.xlsx`, tricking the server into overwriting the confidential marks spreadsheet, resulting in data loss and security breaches.

4. PROTECTING HARDWARE VULNERABILITIES

- Some attacks target the **hardware** instead of the software. These are harder to detect and fix but still important in secure coding.
- **Examples of hardware-based vulnerabilities**
 - CPU caching flaws (e.g., Spectre, Meltdown).
 - Memory access leaks.
 - Physical probing of electronics.
 - Flaws in speculative execution.

HOW TO DEFEND AGAINST HARDWARE VULNERABILITIES

- Keep systems updated with CPU and firmware patches.
 - Separate high-risk tasks into sandboxes or isolated processes.
 - Use memory-safe programming practices.
 - Avoid storing sensitive data in predictable memory locations.
 - Design algorithms to avoid leaking data through timing or CPU behaviour.
- **Relatable example**
 - Two programs run on the same university server. One student writes a program that measures CPU cache timing and uses it to guess the secret key used by the other student's encryption assignment.



PROTECTING HARDWARE VULNERABILITIES

SUMMARY

- Secure code must also account for hardware-level vulnerabilities, where attackers exploit how the device physically processes data—such as CPU caching issues, speculative execution flaws, or memory access patterns—to extract sensitive information. These vulnerabilities may allow attackers to bypass software protections entirely, retrieving passwords, session keys or personal data stored in memory. Developers can reduce hardware-related risks by implementing memory-safe programming practices, isolating sensitive operations, using constant-time algorithms, applying patches for CPU-level vulnerabilities (such as Spectre or Meltdown), and choosing secure hardware configurations. Proper error handling, sandboxing and privilege separation ensure that even if one component is compromised, the attacker cannot access the entire system.
 - **Relatable Example**
 - A shared university server runs multiple student programs. A poorly written program leaks CPU cache access patterns when performing calculations, allowing another student on the same machine to observe memory usage and deduce the secret encryption key used by the server's login system.
-

COMBINED EXAM-STYLE RESPONSE

Safe and secure software must protect against file attacks, side-channel attacks, and hardware-level vulnerabilities to prevent attackers from exploiting unintended system behaviours. File attacks occur when user-controlled file names or paths are mishandled, allowing attackers to upload malicious files or access restricted directories; preventing these issues requires strict file validation, safe storage locations and least-privilege permissions. Side-channel attacks exploit indirect information such as timing or power usage to infer sensitive data, requiring constant-time functions and secure cryptographic libraries. Hardware vulnerabilities occur when attackers exploit CPU or memory behaviour, requiring patches, privilege separation and isolated execution. Together, these protections ensure that software cannot be bypassed through insecure file handling or physical system behaviour.



IMPACT OF SAFE AND SECURE SOFTWARE DEVELOPMENT

COLLABORATION IN DEVELOPING SAFE AND SECURE SOFTWARE

A group of people are gathered around a wooden table in a collaborative workspace. They are working on a project, with a silver laptop open on the table. The table is covered with various items: a large sheet of paper with colorful charts and graphs, several yellow and pink sticky notes with handwritten notes and drawings, a pen holder filled with colored pencils and pens, and a green mug. One person is pointing at a sticky note, while another is writing on it. The background is slightly blurred, showing more people and office furniture.

BENEFITS OF COLLABORATION IN DEVELOPING SAFE AND SECURE SOFTWARE

- Collaboration means working with others to share ideas, divide responsibility and combine different strengths. In software engineering, collaboration is essential for building **safe**, **secure**, and **high-quality** systems. The three major benefits are:
 - **Considering various points of view**
 - **Delegating tasks based on expertise**
 - **Improving the quality of the final solution**
 - Each benefit can also be **applied** in new or unfamiliar development situations, such as building a new school app, designing an online store, or improving an existing cloud system.
-

1. CONSIDERING VARIOUS POINTS OF VIEW

- Considering various points of view means involving people with different roles, backgrounds and experiences when planning and building software. Each person sees risks, weaknesses, usability issues or security gaps that others might miss. Developers might focus on code structure, security specialists look for vulnerabilities, UI designers focus on user mistakes, and end-users identify practical problems. This leads to safer, fairer and more reliable software.
 - When developing a school attendance app, a team could apply this benefit by gathering input from teachers (ease of use), IT staff (security), students (accessibility), and system administrators (data safety). Each viewpoint reveals different security needs—such as session timeouts, password reset features, or visibility of sensitive student information.
-

2. DELEGATING TASKS BASED ON EXPERTISE

- Delegating based on expertise means assigning tasks to the people most qualified to complete them. This ensures complex or security-critical tasks are handled by those with the right skills. For example, a cybersecurity specialist manages authentication logic, while a frontend designer handles interface design. Delegation reduces the chance of mistakes, speeds up development, and ensures the system meets industry security standards.
 - In a ecommerce website project, the team applies this benefit by giving encryption and payment processing tasks to a developer skilled in secure database interactions, while giving UX design to someone familiar with accessibility standards. This division protects sensitive customer data, avoids insecure coding mistakes, and ensures a smooth user experience.
-

3. QUALITY OF THE SOLUTION

- Collaboration improves the quality of the solution because multiple people review the design, test features, find bugs, and evaluate security. Peer review catches vulnerabilities early, group testing strengthens reliability, and shared decision-making produces more thoughtful solutions. A collaborative team also creates clearer documentation, better error handling, and more resilient system architecture.
 - For a mobile banking feature, collaboration can be applied by having developers write the code, testers run penetration tests, security analysts review encryption strength, and designers ensure the interface avoids unsafe user mistakes. This team approach results in a secure, stable and high-quality banking feature that users trust.
-

SUMMARY

Collaboration has clear benefits for developing safe and secure software. Considering various points of view allows different team members—such as developers, testers, end-users and security specialists—to identify risks, usability problems and vulnerabilities that one person alone might miss. Delegating tasks based on expertise ensures that complex or sensitive areas, such as encryption or authentication, are handled by people with the right skills, reducing errors and improving security. Collaboration also improves the overall quality of the final solution because group testing, code review and shared decision-making lead to stronger, safer and more reliable software. Applying these collaborative practices to projects—such as school apps, banking features or ecommerce sites—helps produce systems that better protect users and data.



BENEFITS TO AN ENTERPRISE FROM IMPLEMENTING SAFE AND SECURE DEVELOPMENT PRACTICES

THINKING...

- Safe and secure development practices help enterprises avoid data breaches, protect user information, and maintain trustworthy, reliable systems. When we *investigate* their benefits, we look at how security practices change business operations, and we draw conclusions about their impact. When we *explain* these benefits, we show **why** they happen and the **cause–effect** relationship they create.
-

1. IMPROVED PRODUCTS OR SERVICES

- Safe and secure development leads to software that is more stable, reliable and trustworthy. When we examine organisations that follow strong security practices—such as secure coding standards, regular testing and encryption—we can see that the final products fail less often, expose fewer vulnerabilities and deliver a better experience to customers. The conclusion is that secure development directly improves the overall quality and safety of what the enterprise delivers.
 - Secure development improves products **because** vulnerabilities are identified early, errors are reduced, and systems are harder to exploit. This causes fewer crashes, less downtime and stronger protection of customer data. As a result, customers trust the product more, use it more confidently and are more likely to keep using the service.
-

2. INFLUENCE ON FUTURE SOFTWARE DEVELOPMENT

- Enterprises that commit to safe development practices build long-term habits and frameworks that affect all future projects. When we look into organisations that adopt secure-by-design processes, we find they set coding standards, reuse secure components, and create documentation and guidelines that help future teams avoid past mistakes. The conclusion is that the entire development culture shifts toward security and reliability.
 - Secure development influences future projects **because** teams learn from past vulnerabilities, build libraries of secure code and develop reusable security patterns. This causes future software to become safer from the start, reduces development time and ensures consistent security across all new applications.
-

3. IMPROVED WORK PRACTICES

- Secure development requires clear communication, shared responsibilities and structured processes. Investigating enterprise teams shows that adopting secure coding, code reviews and testing frameworks leads to more organised workflows, better documentation and fewer emergencies. The conclusion is that secure development naturally improves how teams work.
 - Work practices improve **because** secure development encourages teamwork, documentation, planning and consistent coding standards. This leads to fewer rushed fixes, better collaboration between developers and testers, and smoother project management. The effect is a more professional, efficient and predictable work environment.
-

4. PRODUCTIVITY

- By examining enterprises that use secure development practices, we see that although security adds work early in a project, it reduces time spent on fixing problems later. Fewer bugs, fewer breaches and fewer support requests all contribute to better productivity. The conclusion is that secure development saves time in the long run.
 - Productivity increases **because** secure software prevents major incidents, reduces rework and eliminates the need for emergency patches. The effect is that teams spend more time building new features rather than repairing damage, which increases output and reduces costs.
-

5. BUSINESS INTERACTIVITY

- Investigating digitally connected enterprises shows that secure systems allow businesses to safely interact with customers, partners, suppliers and other organisations. Secure APIs, protected data flows and trustworthy authentication systems allow enterprises to integrate with cloud services, online payment systems and partner platforms. The conclusion is that secure development expands business opportunities.
 - Business interactivity improves **because** secure development protects data when systems communicate with each other. This causes organisations to confidently connect to new services, automate processes and collaborate with partners without fear of breaches. The effect is stronger customer engagement, easier integration and more flexible digital operations.
-

SUMMARY

Implementing safe and secure development practices provides major benefits to an enterprise. Products and services improve because secure coding and testing reduce vulnerabilities and create more reliable systems. These practices also influence future development, as teams reuse secure components and follow established standards, leading to safer software long-term. Work practices improve through clearer communication, better documentation and fewer emergencies. Productivity increases because secure systems reduce rework and prevent costly incidents, allowing teams to focus on new features. Finally, business interactivity grows, as secure systems allow safe communication with customers, partners and external platforms. Overall, secure development strengthens every part of enterprise operations through clear cause–effect improvements.

EVALUATING SOCIAL, ETHICAL AND LEGAL RAMIFICATIONS OF SAFE AND SECURE SOFTWARE



THINKING..

- Safe and secure software benefits society by reducing cybercrime, protecting individuals, and supporting stable digital systems. However, introducing strong security can also create social, ethical and legal challenges. To **evaluate**, we weigh both the positive and negative impacts using clear criteria, then make a justified judgement about their overall effect.
-

1. EMPLOYMENT

- Judgement Criteria: impact on workers, enterprise staffing needs, long-term industry change
 - The development of secure software increases demand for skilled cybersecurity professionals, creating new career pathways and improving enterprise resilience. This is positive because it promotes higher-skilled employment and reduces reliance on reactive “firefighting” after breaches. However, improved automation (e.g., automated vulnerability scanners and secure deployment pipelines) can reduce the need for entry-level or low-skill jobs, potentially causing job displacement. Overall, while secure development changes the employment landscape, the long-term judgement is positive because it leads to upskilling, more specialised roles, and stronger workforce capability across industries.
-

2. DATA SECURITY

- **Judgement Criteria: level of protection, risks reduced, consequences if mismanaged**
 - Safe and secure software significantly strengthens data protection for individuals and enterprises, reducing identity theft, fraud, system failures and financial loss. This is a major social and legal benefit because breaches can result in fines, investigations and reputational damage. However, secure systems must be implemented correctly—poor configuration or outdated components can still cause major vulnerabilities. While mistakes remain possible, the overall judgement is strongly positive: secure development practices greatly reduce harm and create safer digital environments.
-

3. PRIVACY

- **Judgement Criteria: user protection, ethical handling of data, possible misuse**
 - Secure software protects privacy by preventing unauthorised access to personal information and supporting ethical data handling. This improves user trust, especially in industries such as health and education where sensitive data is involved. On the other hand, strong security does not automatically stop organisations from collecting excessive data. Some enterprises use secure systems to store large amounts of personal information, raising ethical concerns about surveillance and long-term data retention. Despite these concerns, the overall judgement is positive because security provides the foundation for privacy protection—ethical decision-making must sit on top of secure technical processes.
-

4. COPYRIGHT

- **Judgement Criteria: protection of creative work, fair access, restrictions on users**
 - Secure development helps enterprises enforce copyright laws and protect their digital assets, reducing piracy and safeguarding revenue. This also ensures creators receive credit and compensation for their work. However, strict security systems such as DRM (Digital Rights Management) can limit legitimate user freedoms, such as accessing purchased content on multiple devices or modifying software for accessibility. While security can create restrictions, the overall judgement is balanced but positive—protection of copyrighted works is essential for innovation, provided user rights and accessibility remain considered.
-

5. INTELLECTUAL PROPERTY (IP)

- **Judgement Criteria: protection of innovation, risks of misuse, collaboration impact**
 - Secure software strongly protects intellectual property by preventing the theft of proprietary algorithms, source code, trade secrets or unique business processes. This is beneficial because it encourages enterprises to innovate without fear of losing competitive advantage. However, strict IP enforcement can limit collaboration, especially in open-source communities or joint projects where ownership may be unclear. Despite these drawbacks, the overall judgement is positive: strong security gives enterprises confidence to invest in research and innovation while reducing legal disputes caused by IP theft.
-

6. DIGITAL DISRUPTION

- **Judgement Criteria: industry impact, accessibility, social change, fairness**
 - Secure development enables digital disruption by making new technologies—like cloud computing, AI and online services—safe enough for widespread adoption. This leads to greater efficiency, improved customer experiences and new business models. However, secure digital disruption can widen the gap between large enterprises (which can afford secure upgrades) and small businesses that cannot. It may also cause job loss in industries replaced by secure automated systems. The overall judgement is mixed but leans positive: digital disruption delivers major benefits, but enterprises must support workers and small organisations to minimise negative impacts.
-

SUMMARY

The development and implementation of safe and secure software create a mix of social, ethical and legal ramifications for people and enterprises. Employment generally benefits through the creation of high-skill cybersecurity roles, though automation may reduce some low-skill positions. Data security improves significantly, reducing breaches and financial harm, but relies on correct implementation. Privacy protection increases because secure systems reduce unauthorised access, although organisations may still over-collect data, raising ethical concerns. Copyright and intellectual property are better protected, supporting innovation, but sometimes restrict user freedoms and collaboration. Digital disruption allows enterprises to adopt new technologies safely, improving efficiency, although it can disadvantage smaller businesses and displace some workers. Overall, secure software development has mostly positive impacts, but enterprises must manage the ethical and social consequences to ensure fair and responsible use.

HSC STYLE QUESTIONS



EXPLAIN HOW CONFIDENTIALITY AND INTEGRITY CAN BE MAINTAINED WHEN DEVELOPING A SCHOOL DATABASE APPLICATION. (4 MARKS)

Confidentiality can be maintained in a school database application by ensuring that student records are accessible only to authorised staff. This can be achieved through encryption of sensitive information such as addresses, contact details, and medical data, as well as enforcing secure logins for teachers and administrators. Role-based access control ensures that users see only the information relevant to their responsibilities. Integrity is maintained by validating all data entered into the system, preventing errors or malicious alterations. Techniques such as database constraints, input sanitisation, and transaction controls help ensure that stored records remain accurate and consistent over time. Together, these measures protect both the privacy and reliability of student information.

EXPLAIN HOW AUTHENTICATION, AUTHORISATION, AND ACCOUNTABILITY WORK TOGETHER TO PROTECT A CLOUD-BASED FILE-SHARING SYSTEM. (6 MARKS)

In a cloud-based file-sharing system, authentication, authorisation, and accountability form a layered defence against unauthorised access and misuse. Authentication verifies user identity using secure credentials such as usernames, passwords, or multi-factor authentication. Once authenticated, authorisation determines what actions a user is permitted to perform — for instance, some users may upload and edit files, while others can only view or download them. Accountability ensures all actions are traceable through activity logs and audit trails, allowing administrators to monitor who accessed, modified, or shared files. These three mechanisms work together to confirm user identity, restrict access to appropriate resources, and record activity for transparency and investigation, thereby ensuring the security and integrity of the file-sharing environment.

EXPLAIN HOW THE SIX CORE
SECURITY CONCEPTS —
CONFIDENTIALITY, INTEGRITY,
AVAILABILITY,
AUTHENTICATION,
AUTHORISATION, AND
ACCOUNTABILITY — CAN BE
APPLIED WHEN DEVELOPING A
SECURE E-COMMERCE
PLATFORM.



In a secure e-commerce platform, security begins with **authentication**, verifying user identity through secure logins or two-factor verification. Once verified, **authorisation** determines what the user can do — customers can view and purchase products, while administrators can manage inventory. To maintain **confidentiality**, sensitive information such as payment details is encrypted and only visible to authorised users. **Integrity** ensures that transactions are accurate and cannot be tampered with through validation checks and secure APIs. **Availability** is maintained through reliable servers, backups, and DDoS protection, ensuring customers can access the site at any time. Finally, **accountability** is upheld through detailed logging and monitoring of transactions, allowing administrators to trace activity and detect fraud. Together, these six principles establish a strong, layered security foundation for the e-commerce system.